A Web-Based Bicycle Repair Management System For Performsport

Group 14
Anders Mathias Larsen, Daniel Sloth, Jacob Christian Larsen, Philip Vestergaard Hellerup Jørgensen, Viktor Alexander Parkhøi, Zelalem Niguse Gebremeskel and Rasmus Juul Severinsen

STUDENT REPORT
AALBORG UNIVERSITY

AALBORG UNIVERSITY
STUDENT REPORT

Computer Science
Aalborg University
http://cs.aau.dk

Title:
A Web-Based Bicycle Repair Management System For Performsport

Theme:
Improving Bike-Repair Shop Efficiency Through Digital Scheduling Tools

Project Period:
Fall Semester 2025

Project Group:
Group 14

Participants:
Anders Mathias Larsen
Daniel Sloth
Jacob Christian Larsen
Philip Vestergaard Hellerup Jørgensen
Viktor Alexander Parkhøi
Zelalem Niguse Gebremeskel
Rasmus Juul Severinsen

Supervisor:
Carsten Ruseng Jakobsen

Date:
2026-08-19

Copies: 1

Page Numbers: 74

Abstract:
The report presents the development of a web-based repair management system for the bicycle shop Performsport, designed to replace their manual paper-based workflow with a digital platform. The system is implemented using Spring Boot, PostgreSQL, Thymeleaf, and Docker, and provides employees with tools to create, update, and track repairs through an interactive calendar and list view interface.

A layered software architecture was applied, separating the application into controllers, services, repositories, and domain entities to ensure clarity, maintainability, and scalability throughout development. The platform allows staff to assign parts and services to repairs, adjust quantities, and calculate total costs based on registered labor and spare parts. System architecture, design decisions, implementation details, and testing are documented to evaluate the solution's functionality and performance. The final product demonstrates a functioning system that streamlines repair handling, improves workflow efficiency, and provides a foundation for continued development.

Contents

Preface

This project is produced by group cs-25-sw-3-14, a 3rd semester software engineering group from Aalborg University. During this semester, the group was tasked with developing a well-structured application as part of the program theme focused on software architecture, design patterns, object-oriented analysis and programming, and modern development practices. The project builds on the skills and knowledge acquired during the first two semesters and places emphasis on requirements analysis, architectural design, systematic testing, and reflection on the development process.

The result is a repair-management web application developed for Performsport, a local cycling shop in Aalborg. The system supports repair handling, scheduling, and the management of parts and services, and is built using a multi-layered architecture with Spring Boot, PostgreSQL, and Docker. The project includes requirements modelling in the object-oriented paradigm, architectural design using established patterns, implementation, integration testing, and evaluation of the user interface. The work was carried out with direct involvement of an external client and completed under the supervision of Carsten Ruseng Jakobsen from Aalborg University.

The final implementation and source code are included in the project attachments and are available at: https://github.com/PhilipHellerup/P3-Project-Program

Anders Mathias Larsen
amla24@student.aau.dk

Daniel Sloth
dsloth23@student.aau.dk

Jacob Christian Larsen
jcla24@student.aau.dk

Philip Vestergaard Hellerup Jørgensen
pjorge24@student.aau.dk

Viktor Alexander Parkhøi
vparkh24@student.aau.dk

Zelalem Niguse Gebremeskel
zgebre24@student.aau.dk

Rasmus Juul Severinsen
rsever22@student.aau.dk

1 Introduction

Digital tools have become an essential part of how modern businesses organize their work. When information is collected in one place instead of scattered across notes, emails, or poorly designed systems, it becomes easier for employees to maintain an overview, coordinate tasks, and avoid misunderstandings. Well-designed digital workflows not only make everyday routines smoother but also enable organizations to respond more quickly when plans change or new tasks arise.

At Performsport, repairs are currently handled through a mix of handwritten notes and draft orders in Shopify. Although this method works in practice, it does not offer a clear and consistent overview of ongoing repair jobs. Mechanics and sales staff often need to look in several different places to find information, making it difficult to keep track of the progress of a scheduled repair. These challenges highlighted the need for a digital system that centralizes repair information, enabling both sales personnel and mechanics to access the necessary data from a single location.

This project explores how such a solution can be designed and implemented. The goal is to develop a web-based repair management system that aligns with Performsport's daily workflow, making it easier to plan repairs, track their status, and add relevant parts or services. The system combines a calendar for scheduling, a list view for quick access to details, and simple tools for updating each repair as work progresses. The solution is based on object-oriented principles, structured architectural design, and proven frameworks, such as Spring Boot, Thymeleaf, and FullCalendar.

The report documents the full development process, from the initial analysis of Performsport's workflow to the definition of requirements and the design and implementation of the final system. It also includes the results of integration, unit, and user testing, as well as an evaluation of how well the system meets the identified needs, followed by a discussion of improvements that could be developed in the future.

2 System Description

This chapter introduces the collaboration with sports retailer Performsport and describes how their current systems work and which problems they encounter with this system. This is explored using a rich picture drawing and the FACTOR criterion. From this, requirements are defined for a new system to streamline the bike-repair process.

2.1 Intro to Performsport

Performsport is a sports retailer in Aalborg and Holstebro that specializes in running, cycling, and multi-sport training gear. Performsport sells bikes, clothing, and other sports items in two physical locations as well as through an online store. In addition to selling equipment and clothing, the company also offers services such as bike repairs, bike fitting, and running analysis.

After contacting the business, they described that they are looking for a calendar system for planning bike repairs internally. A meeting was scheduled to discuss Performsport's current system, the requirements for a new system, and how such a solution could be developed to fit their needs. The meeting was set up as a semi-structured interview, using a set list of questions that needed to be answered while also allowing the conversation to drift, so more time could be spent on the points most important to Performsport.

A description of Performsport's current systems and requirements is derived from this conversation and is described in the following sections. For the interview notes, refer to Section 13.1.

2.1.1 Current System

Online and in-store sales at Performsport are managed through Shopify[1], using both its e-commerce platform and Point of Sale (POS) system, which is a combination of hardware and software used to process customer transactions at a physical store location. Appointments for services such as bike fitting, physio, and running analysis are booked via a Shopify calendar application, which is integrated into the Performsport website for online scheduling. While this calendar tool works well for these types of services, it lacks the necessary functionality to schedule bike repairs.

As a workaround, Performsport currently relies on paper notes combined with Shopify to manage bike repairs. When a customer drops off their bike, an employee creates a draft order in Shopify with the customer's details and repair specifications. A draft order allows Performsport to add products and services to an order until they are ready to take payment from the customer. This draft order is then printed and attached to the bike, which is stored in the back room until a mechanic has time to repair it. Mechanics use these notes to track a single repair, however, this system provides little structure for the overall repair workflow. For example, it is difficult to see which bikes are waiting for repair, are currently being repaired, or are delayed while

[1]Shopify is an all-in-one e-commerce platform, which allows individual businesses to host an online store [1]

waiting for parts. It is also difficult for a salesperson to know when to schedule a new repair, as there is no place to see which repairs are scheduled for each day.

Once a repair is finished, the mechanic updates the draft order in Shopify with all costs, including labor and spare parts. The draft order is then ready for payment when the customer picks up their bike.

2.1.2 Rich Picture

Figure 1 shows the rich picture visualizing this workflow. A customer interacts with a Performsport employee when dropping off and collecting their bike. The employee manages the process by entering repair details in Shopify and preparing a physical note to attach to the bike. Mechanics then rely on both the note and Shopify to gather repair information, resulting in information being spread across two sources.

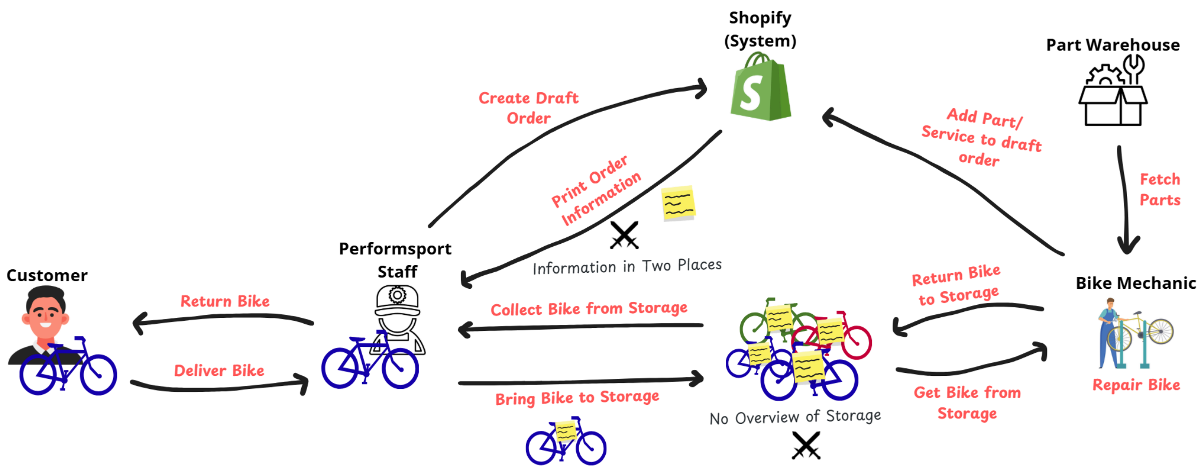


Figure 1: Rich Picture Drawing of Performsport's Current System

2.1.3 Performsport's Problem

Performsport's current approach to handling repairs is inefficient and fragmented, as information is duplicated across Shopify and paper notes, leaving mechanics without a clear overview of the repair queue. The lack of centralized tracking makes it difficult to monitor repair status, prioritize tasks, or follow up on bikes awaiting parts.

Performsport is therefore seeking a custom solution that:

- Provides an overview of all bikes in the shop within a calendar or scheduling view.
- Displays essential information for each repair, such as bike type, customer details, and repair status.
- Possibly integrates with Shopify, so that spare parts can be added directly to a repair job, and a draft order is automatically generated once the repair is complete.

This solution would streamline the repair workflow, enhance visibility for mechanics and employees, and reduce the inefficiencies caused by the current paper-based system. This description leads to the following problem statement.

2.2 Problem Statement

The project aims to develop a web application for Performsport that centralizes repair scheduling in a calendar system, where new repair jobs can easily be created and made visible to both staff and mechanics.

2.3 System Description Using the FACTOR Criterion

The FACTOR criteria are used as a framework to describe how a system based on Performsport's requirements might look. It focuses on six key elements, evaluating them to identify essential functionalities, when and where the system is used, and by whom. These insights are then used to formulate the precise requirements for the system.

Functionality:
Employees at Performsport are able to organize repair jobs by scheduling them in a calendar application. Employees can add, edit, delete, and move around repair jobs in the calendar to fit their schedule. For each repair, they can see the type of job, e.g., a bike service or repair, bike type, and customer information. Mechanics can add the parts they use for the repair on each job.

Employees can see the status of a repair on the calendar entries to get a quick overview of which bikes are in the shop and how far in the repair process each bike is.

Application Domain:
The system is used on the web by Performsport sales personnel and mechanics on their personal computers, as well as an in-store shared desktop computer. Sales personnel and mechanics have the same permissions and access to the same information. They will use the system in two different ways. The sales personnel will use the system to create repairs and take payment, while the mechanics uses it to view information about a repair and add parts and services to a repair.

Conditions:
The application will be developed by a group of computer science students from Aalborg University in collaboration with Performsport, which must satisfy the requirements of the semester project. Due to the application being developed as a semester project, the project scope is limited to approximately four months. The requirements are set based on this scope.

The application will be used daily by Performsport's staff during opening hours for the shop to manage and schedule bike repair jobs. Sales personnel will create and organize new repair jobs when customers drop off their bikes or via a phone call to schedule a drop off. Mechanics will use the same system throughout the day to track, update, and complete ongoing repairs. The system's use case is therefore to support coordination between front-desk staff and mechanics to ensure smooth workflow and clear job visibility.

Technology:
The system will be a web application available to employees of Performsport, which will be written in Java with Spring[2], using an object-oriented method. The web application will be run and accessed via a desktop PC.

The system will run on desktop computers located at Performsport's physical stores. It will be accessible through a web browser and hosted locally or on an internal server. Since the system is designed for in-store use by employees, it is optimized for desktop screens and standard input

[2]Java-based framework to create web applications and micro services[2]

devices such as keyboard and mouse. Mobile and tablet versions are not within the current project scope.

Object:
Primary entities in the problem domain are employees, customers, and repairs. In relation to bike repairs, employees can act as sales personnel, mechanics, or both. The system does not distinguish between employee roles, all employees will have access to the same information, and the application will function identically for both sales staff and mechanics. Customers can be imported from Performsport's existing Shopify database, but this requires Shopify integration, which is not the system's top priority. A repair refers to any service, repair, or other task performed on a bike by a mechanic. Each repair will store information about the customer, the bike, the type of service, and the repair's status (e.g., not received, received, finished).

Responsibility:
The web application will serve as an administrative tool for Performsport employees, providing an overview and control over the incoming and current bike repairs. Its purpose is to support a more efficient and intuitive workflow across the company, instead of the current paper-based repair management process, which causes inefficiency, a lack of overview, and duplicated information between their paper notes and Shopify. The new system centralizes all repair jobs in a digital calendar, giving a clearer overview and streamlined workflow for employees.

2.4 Requirements Using MoSCoW

Requirements are based on the interview with Performsport, the rich picture, and the FACTOR criterion. Requirements are set up using the MoSCoW method, ranking requirements as must-have, should-have, could-have, and will-not-have, based on the significance of requirements [3].

Must-have

- **Option to Create a New Repair**
  An employee should easily be able to create a new job/repair containing: bike brand and model, customer information, type of repair job (e.g., a service, repair, or adjustment), notes, and date/time. The repair should then be added to the calendar from the specified date and time of the repair.
- **Show Repairs in a Calendar**
  Repairs should be shown as a block in a calendar, with the most important repair information, like bike name, type of repair, and customer contact details, being visible on each repair. The application must be able to show the calendar in a day, week, and month view.
- **Show Repairs as a List**
  Repairs should also be shown as a list in which the mechanic can search for a specific repair job using the customer's phone number, name, or bike type. This should also include former repairs that have been completed.
- **Show the Status of Each Repair**
  Each repair should have a status marker, showing if the bike is: not received, received, under repair, or finished. There should also be a status option "waiting for spare part", for when a repair needs a special item or when a part is not in stock.
- **Edit, Move & Delete Repairs**
  Employees should be able to move, delete, and edit bike and customer information for a repair from both the calendar and the list view.

- **Add Labor Cost to a Repair**
  Mechanics should be able to specify the time used for a repair, and the total cost of labor should be calculated and shown on the repair.
- **Add Spare Part to a Repair**
  Employees should have an option to add a part to a repair, such as a new chain for the cassette on the bike. There should also be an option to edit or delete a part. A cost for the part should be added as well.

Should-have

- **A Predefined List of Services**
  When creating a repair, employees should be able to choose from a list of predefined services, automatically calculating an estimate for how long a repair will take and how much it costs. Performsport should be able to edit and add new types of services in a separate menu.
- **Color Coded Repairs**
  Repairs should be colored based on the status of the repair. E.g., when a bike has been received, it should change color, letting the employees know that the bike is in the store and ready.

Could-have

- **Add Discounts to a Repair**
  Employees should be able to add a discount to a repair and any spare part included in the repair.
- **Integration of Existing Shopify Solution**
  The Application should be integrated with Performsport's existing Shopify solution. This includes integration with their customer database, allowing products to be added to a repair, inventory to be looked up for parts and modified, and the creation of draft orders to automatically generate a draft order for the repair upon completion.
- **Smart Part Search**
  When searching for a spare part, the list should highlight the items that fit on the specified bike, e.g., the length of the chain or cassette.
- **Use Application on Mobile**
  The application's UI should be scaled to be usable to work on mobile phones.
- **Mark a Repair as a Warranty Case**
  There should be an option to mark the repair job as a warranty case, and e.g., change the price or apply a discount to the repair job.

Will-not-have

- **Store customer information**
  Store information on users to enable reuse on multiple repairs
- **Online Booking**
  Customers can schedule repairs online on the Performsport website.

2.5 System Definition Summary

Based on the conducted FACTOR analysis and the prioritization of requirements set by Performsport, a system to streamline the current bike repair workflow can be defined.

The system will provide a web-based calendar that centralizes repair scheduling, tracks repair statuses, and gives both mechanics and sales staff a clear overview of ongoing repairs. While integration with Shopify would be desirable, it may fall outside the scope of this project.

To meet Performsport's expectations, the application must fulfill the defined must-have requirements in the MoSCoW section above in Section 2.4. This includes the ability to create, edit, delete, and move repairs within the calendar. The application should also track the parts used for each repair and, potentially, integrate this functionality with Shopify's existing inventory management.

The next chapter describes which classes and events interact to structure the system.

3 Problem Domain Analysis

The purpose of this chapter is to model Performsport's real-world environment when using the new calendar system. It begins by outlining the core classes and events that are essential for the system to function, together with an explanation of how these elements interact within the workflow of bike repairs. By identifying the key entities, such as repairs, services, and parts, and describing their relationships, the problem domain provides a structured view of the information that the system must handle.

To illustrate this, the chapter presents three models. An event diagram, which captures the main events that occur in the repair process, a class diagram, which shows the system's structure through its classes, attributes, and associations, and behavioral diagrams, which explain how the system behaves dynamically when events and classes interact. Together, these diagrams form the foundation for designing the new application and ensure that the system aligns with Performsport's requirements.

3.1 Classes

Each class describes a selection of objects in the problem domain and stores essential information for the system. Table 1 shows the main classes used in a bike repair scheduling application for Performsport, and a description of what information each class stores.

Class	**Description**
Repair	The repair class contains all necessary information about each repair, e.g., customer information, parts used, and labor cost, etc.
Service	The service class contains predefined services, which can be added to a repair in the creation process. These contain a title, price, and a duration.
Part	The part class contains predefined parts, which can be added to a repair, with a title and a price.

Table 1: Table of Classes in the Problem Domain with Descriptions

A separate customer class could be added to enable the reuse of customer information across multiple repairs. However, since Performsport already maintains a user database in their Shopify solution, this functionality is not considered essential in the current system, as of Section 2.4. As customer information is only relevant within the context of a repair, it is stored directly in each repair.

Using these classes, the system has information about all essential parts of the problem domain. The next section covers the events that occur in the problem domain, which are used together with the classes to model the structure of the program.

3.2 Events

Each event corresponds to an action performed by a Performsport employee, from creating a repair to updating a repair status to notify the customer when a repair is completed. Table 2 shows events performed by employees during a bike repair flow, and when managing predefined parts and services:

Events	**Description**
Repair scheduled	A new repair job is scheduled on the calendar. The repair includes customer details, bike description, and repair type.
Repair cancelled	An existing repair is canceled, and the associated calendar entry is cleared.
Repair rescheduled	The date and time for an existing repair job are changed, moving it to a new position in the calendar.
Repair edited	The information in an existing repair job is changed.
Service added to repair	A predefined service is assigned to a specific repair job, automatically updating the price and duration of the repair.
Part added to repair	A predefined part is assigned to a specific repair job, automatically updating the price of the repair.
Repair status changed	The status of a repair (e.g., in progress or finished) is updated by an employee, and the calendar is updated accordingly.
Defined new part	A part is defined in the part list.
Edit part	Information for a part in the parts list has been edited.
Part discontinued	A part from the part list has been discontinued, hence no longer a part of the part list.
Defined new service	A service is defined in the service list.
Edit service	A service from the service list has been edited, including changes to its title, cost, and duration.
Service discontinued	A service from the service list has been discontinued, hence no longer a part of the service list.

Table 2: Table of Events in the Problem Domain with Descriptions

The next section combines classes and events in an event table to give a better understanding of which objects are part of which event.

3.3 Event Table

An event table is a tool used to model the problem domain by showing the relationship between selected classes and the events that involve them. In Table 3, each column lists the chosen classes, and each row lists the selected events. A "+" in a cell indicates that objects from the corresponding class are involved in that specific event at most one time. A "*" shows that a class can be involved in an event multiple times [4].

When a new repair is scheduled, the system needs information about the parts and services associated with a repair. This can happen once for each repair object and multiple times for a part or service.

Adding a new part/service to a repair uses information about which repair to add it to, and attributes such as the price of a product. These events can happen multiple times for both the repair, part, and service object.

Defining or discontinuing predefined parts and services only requires information about which part or service is being modified. This can only happen for each part and service once.

Event / Class	**Repair**	**Service**	**Part**
Repair scheduled	+	*	*
Repair cancelled	+		
Repair rescheduled	*		
Repair edited	*		
Service Added to repair	*	*	
Part added to repair	*		*
Repair status changed	*		
Defined new part			+
Part Discontinued			+
Defined new service		+	
Service Discontinued		+	

Table 3: Event Table for Events Described in Section 3.2

Table 3 is an event table used to create a class diagram in the next section, showing how classes are related based on the relations shown in the event table. This gives a sense of the overall structure of the system.

3.4 Structure

The system structure is described using a class diagram in Figure 2. The class diagram shows how the classes are related through aggregation and generalization, and also the cardinalities of these relations.

The Product class serves as a superclass that generalizes both the Service and Part classes, as they share core attributes such as a title and price. In this structure, services and parts have common traits and similar roles in the repair process, and both contribute to calculating the total cost of a repair. Grouping them under a shared superclass avoids unnecessary repetition and aligns with object-oriented principles of abstraction and inheritance, where common data and behavior are placed in a higher-level class. The abstract Product class, therefore, functions as the superclass for all items that can be added to a repair, rather than using an interface.

The diamond shape indicates an aggregation relationship. A Repair consists of zero or more Parts and Services, and each Part or Service may be associated with zero or more Repairs. In this model, each Part object represents a type of part rather than a unique physical item. It only records the cost and the fact that a part of that type was used.

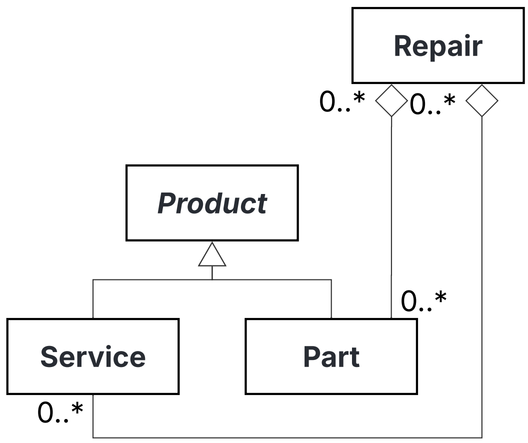


Figure 2: Problem Domain Structure Shown as a Class Diagram

Figure 2 is evaluated based on how effectively it provides users with the information they need. To evaluate this, a set of scenarios is created and tested to determine whether users can access the required information. If a scenario reveals missing or unclear information, the class diagram is adjusted accordingly.

For example, consider a situation where a mechanic communicated with a supplier. The mechanic might need to identify which parts to order for the upcoming week. By reviewing the scheduled repairs, the mechanic can determine and order the necessary parts in advance.

Similarly, when a customer comes to pick up their bike, a salesperson can calculate the repair cost by referring to the parts and services involved, using the prices defined as attributes within the product superclass.

The class diagram outlines the structural relationships between products, repairs, days, and the calendar, capturing the static perspective. The next section complements this by focusing on behavior, showing how key classes evolve and respond to events.

3.5 Behavior

This section describes the behavior of the most important classes in the system. The behavior of classes is modeled using behavior diagrams, which show the life cycle of a class and which event occurs when it exists. Figure 3 shows the life cycle of a repair object. After a new repair is scheduled, a user can add a part or service, change the state of the repair, e.g., *delivered*, *in progress*, or *missing part*, or reschedule a repair. These events can modify a repair object until it is set as *finished* and paid for or removed.

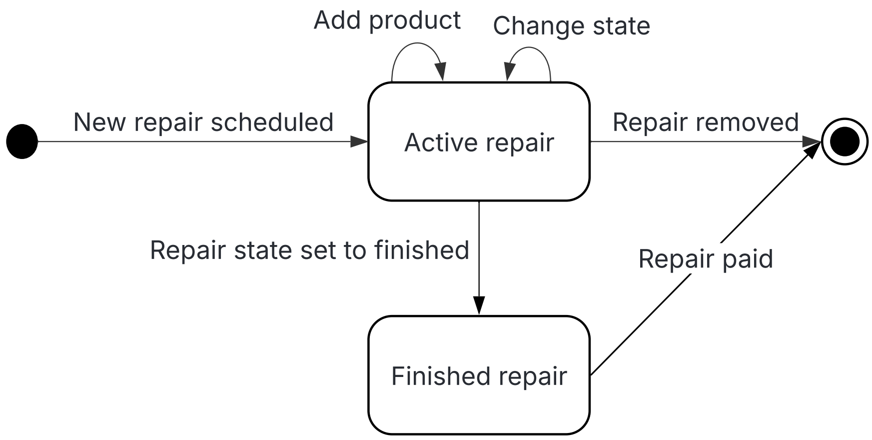


Figure 3: Behavior of the Repair Class as a Behavior Diagram

The behavior of the Part and Service classes is very similar in the program. They can both be used for a repair and be edited to match any changes to their price or availability. Their behavior is modeled below through their *Product* superclass in Figure 4. The product life cycle starts when defined by a Performsport employee and ends once the product is discontinued.

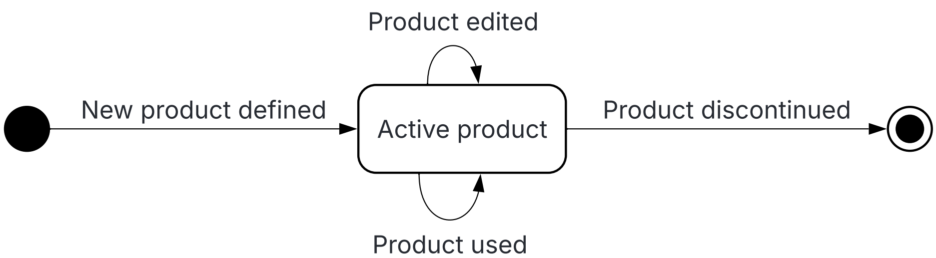


Figure 4: Behavior of the Product Class as a Behavior Diagram

The problem domain describes the real-world objects at Performsport, and which events the system has to model. The next chapter looks at the people using the program and in which way they intend to use it.

4 Application Domain Analysis

An application domain analysis serves to determine how a system is intended to be used in practice. This involves examining the system's usage context and identifying the various actors that interact with it. Furthermore, the system's functions are analyzed to define the interface elements required, ensuring that users can engage effectively with the system.

4.1 Actors & Use Cases

Based on the system definition in Section 2, a set of relevant actors and corresponding use cases is derived. These are then organized and presented in an actor table.

The application domain for this system has two main actors. The sales personnel, who interact with customers and create repairs, and the bike mechanic, who does the repairs. Performsport has cases where one person acts as both a salesperson and a mechanic, which have different use cases for the system. These are listed in the table below, where an "X" indicates which actors are associated with each use case.

Use Cases / Actors	**Sales Personnel**	**Mechanic**
Create a new repair	X	
View repair diagnostics		X
Add a service to a repair	X	X
Add a part to a repair	X	X
Create invoice from repair for POS system	X	

Table 4: Table of Use-Cases From the Application Domain

Section 4.2 explains how each use case is realized in the system, using state chart diagrams to illustrate the system's state during a use case.

4.2 State Chart Diagrams

State chart diagrams illustrate how the system operates in each scenario from Section 4.1, highlighting the actions performed by the actor throughout the process. After each user action, the system transitions to a new state, presenting the user with new information.

Th e first use case of the application is *Create Repair*, which involves the initial registration of a repair job in the system. This represents one of the core interactions users will have with the system and is therefore an essential requirement. After the user creates a new repair, they progressively fill out repair information and add services until they set an initial state and finish the repair job creation.

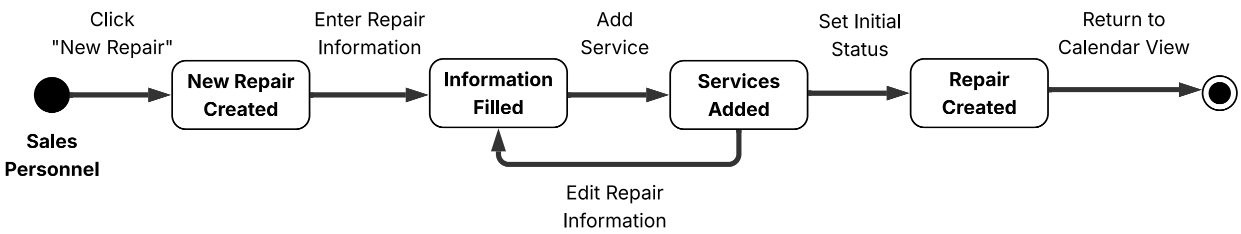


Figure 5: Illustration of the *Create New Repair* Use-Case as a State Chart Diagram

To begin the repair job, mechanics must retrieve the repair diagnostics documented in the system. The initial type of repair is either diagnosed by a mechanic or specified by the customer when the repair job is created.

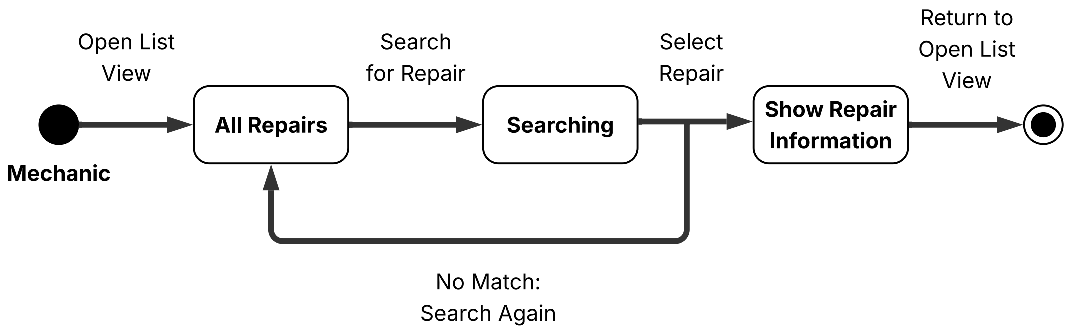


Figure 6: Illustration of the *Retrieve Repair Diagnostic* Use-Case as a State Chart Diagram

During the repair or at the initial repair job creation, a type of service can be specified. The type of service will be a class containing information, such as the type and duration.

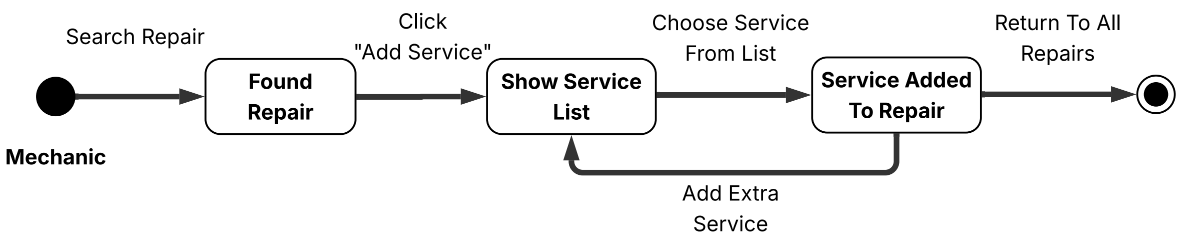


Figure 7: Illustration of the *Add Service(s) to a Repair* Use-Case as a State Chart Diagram

During the repair or at the initial repair job creation, parts can be added to the repair. The part class will contain information, such as type and price.

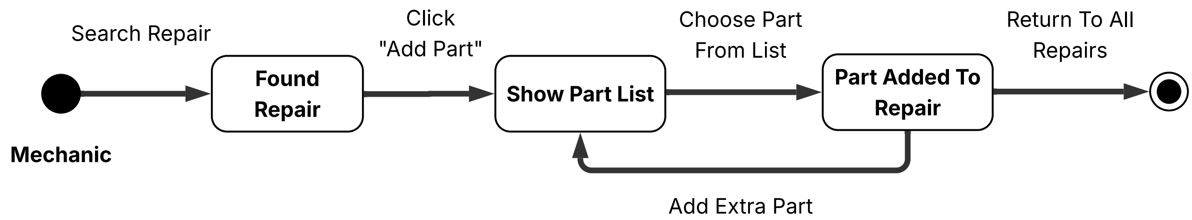


Figure 8: Illustration of the *Add Part(s) to a Repair* Use-Case as a State Chart Diagram

Upon completion of the repair, an invoice is created, enabling sales personnel to charge the customer accordingly.

Sales personnel will then send a notification after receiving information about the completion of a repair from a mechanic. Customers will receive said notification.

4.3 System Functions

The above sections have described the actors in the application domain and their use cases of the system. To assist users in their work, the system must have a set of functions that gives users access to the required information or change the state of objects in the problem domain. These are the functions that enable actors to perform a use case using the system. Functions are identified partly from classes and events in the problem domain description in Section 3 and the use cases in the application domain. Functions are split into tables based on where the system utilizes them. Each function is listed with its complexity and type.

The complexity and type of each function were determined using the guidelines described in [4].

Complexity reflects the expected effort and technical difficulty of implementing a function, depending on the number of involved classes, the amount of logic, and the degree of coordination required between objects.

- **Simple functions** involve one or a few classes and limited logic, such as reading or updating a single object.
- **Medium functions** require moderate logic or interaction between multiple objects, such as searching for a repair or moving a repair.
- **Complex functions** involve several objects, dependencies, or significant processing, such as generating or updating calendar views.

For future testing of the system, there will be a bigger emphasis on testing functions which are of the complex type.

The type expresses a relation between the model and the system's context and has characteristics that help in defining the function types. The book [4] identifies four main types of functions, but only three of the four types are utilized; hence, only the three function types will be explained:

- **Read functions** are triggered by a need for information in an actor's work task and result in the system displaying relevant parts of the model.

- **Update functions** are initiated by a problem domain event and result in a change in the model's state.
- **Compute functions** are activated by a need for information in an actor's work task and consist of a computation involving information provided by the actor or the model. The result is a display of the computation's result.

Each function in the tables was therefore assigned a type and complexity according to these criteria, based on its purpose in the use cases and its interaction with the problem domain model.

Functions to handle the calendar are listed in Table 5 below. The calendar functions allow users to view repair jobs in a calendar generated from the information stored in each Repair object. Users can also change the view mode to display repair jobs for a specific day, week, or month.

Function	Complexity	Type
Show repairs as a list view	Simple	Read
Search for a repair	Medium	Read
Show repairs as calendar view	Complex	Read
Move a repair between days	Medium	Update
Change view mode on calendar	Complex	Read
Create a new repair	Complex	Update

Table 5: Table of Calendar Functions with Complexity and type

Functions for managing the information and state of each repair are shown in Table 6 below. To enable actors to perform use cases from the application domain, they must be able to change information on a repair, add parts and services, and change the status of a repair. For sales personnel to handle payments at Performsport's POS system, the price of a repair must be calculated using the price of added parts and services. For mechanics to plan work days, the system must calculate the estimated time needed for each repair. Using this calculated time, mechanics know whether they can fit new repair jobs into a given day.

Function	Complexity	Type
Change customer information on repair	Simple	Update
Add part to repair	Simple	Update
Add service to repair	Simple	Update
Change status on repair	Medium	Update
Calculate price	Simple	Compute
Calculate repair duration	Simple	Compute
Delete repair	Medium	Update

Table 6: Table of Repair Functions with Complexity and type

In addition to managing repair jobs and handling the calendar, the system also allows employees to maintain the internal lists of services and spare parts. These functions ensure that mechanics and sales personnel can easily update the resources used in repairs.

Function	Complexity	Type
Add new part to product list	Simple	Update
Edit existing part or service	Simple	Update
Remove part or service from list	Medium	Update
Search for parts or services	Simple	Read
Filter product list by category (part/service)	Simple	Read

Table 7: Table of Inventory Functions with Complexity and type

In this case, complex functions are changing the view mode of the calendar, creating a new instance of the repair class with the correct information, and storing it in the database. Complexity is estimated using the definitions proposed above. Creating a repair is complex because it involves several objects, and changing the calendar view mode is complex due to the front-end layout changes and correctly loading repairs.

These functions define the system's core capabilities and how users interact with it. Each function corresponds to specific user actions and is accessed through the system's interface. While this chapter focused on what the system does, the next chapter addresses how these functions are presented to users. The User Interface Design chapter outlines the layout and interaction design that enables users to perform their tasks efficiently and intuitively.

5 User Interface Design

This section will examine the key principles, patterns, and considerations that make an effective UI design. Visual hierarchy, consistency, and accessibility play a crucial role in shaping usability and the overall user experience [5]. In this context, wire framing serves as a particularly effective method for creating visual prototypes for interactive systems.

5.1 Lo-Fi Wireframes

Lo-fi wireframes are simple prototypes emphasizing layout and functionality. They allow designers to explore interaction flows and identify usability issues early in the design process [5]. Specifically, the lo-fi wireframes included here focus on three core elements of the system: a *Calendar* for scheduling repairs and general time management, a *Repair/Job List* window for efficient appointment search, and a *Product* window for managing predefined products.

5.1.1 Lo-Fi Calendar

The wireframe in Figure 9 represents the *Calendar* page, where all scheduled repairs and services are displayed. The layout is divided into a left-hand side menu and a main content area. The side menu provides quick navigation to the three core sections of the application: *Calendar*, *Job List*, and *Products*. Each calendar entry includes important details, such as the type of bike, customer information, and any parts used in the repair or service. Users can switch calendar view between day, week, and month to gain the level of detail or overview that best suits user needs and preferences.

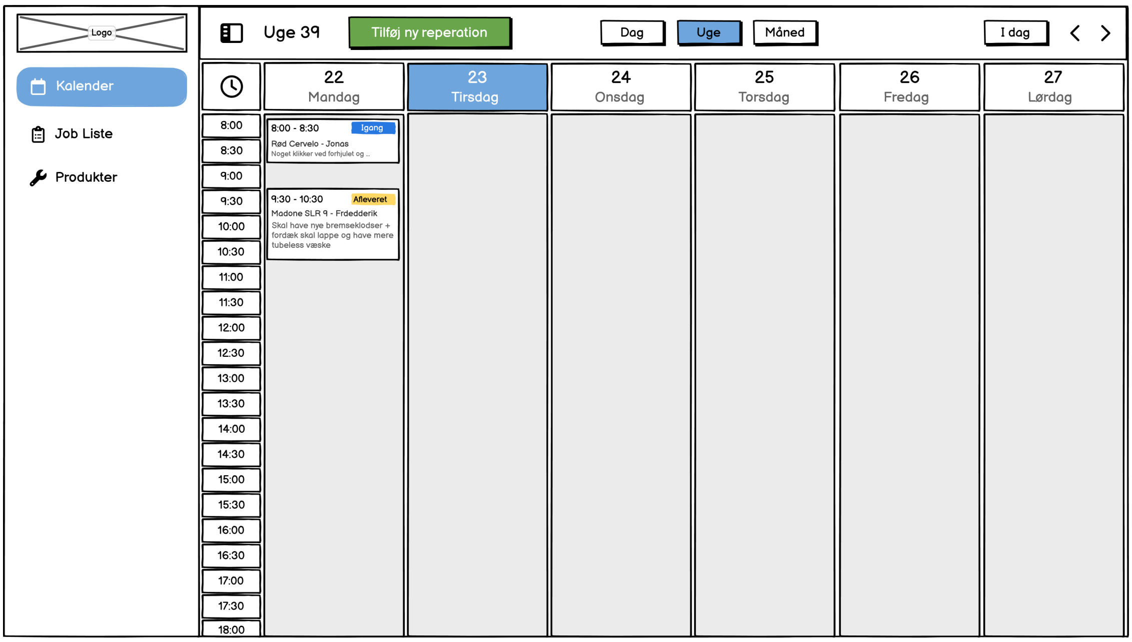


Figure 9: Lo-Fi Calendar Wireframe

5.1.2 Lo-Fi Job List

Figure Figure 10 presents the low-fidelity wireframe for the Job List page. This view provides a structured overview of all repair orders currently in the system, arranged in columns showing order details, repair status, description, and scheduled time.

At the top, toggle buttons allow staff to switch between Active and Completed repairs, making it easy to focus on the relevant set of tasks. A search field supports quick look-ups by name, phone number, or bike type, while a filter icon offers additional options for narrowing the list.

Each repair entry displays key customer information, such as name, phone number, and bicycle model, along with a clear status label (e.g., *In Progress*, *Finished*, *Collected*). This design helps employees track progress and maintain a quick understanding of ongoing and completed repairs. A notable "Add New Repair" button, marked in green, enables the fast creation of new repairs directly from this page.

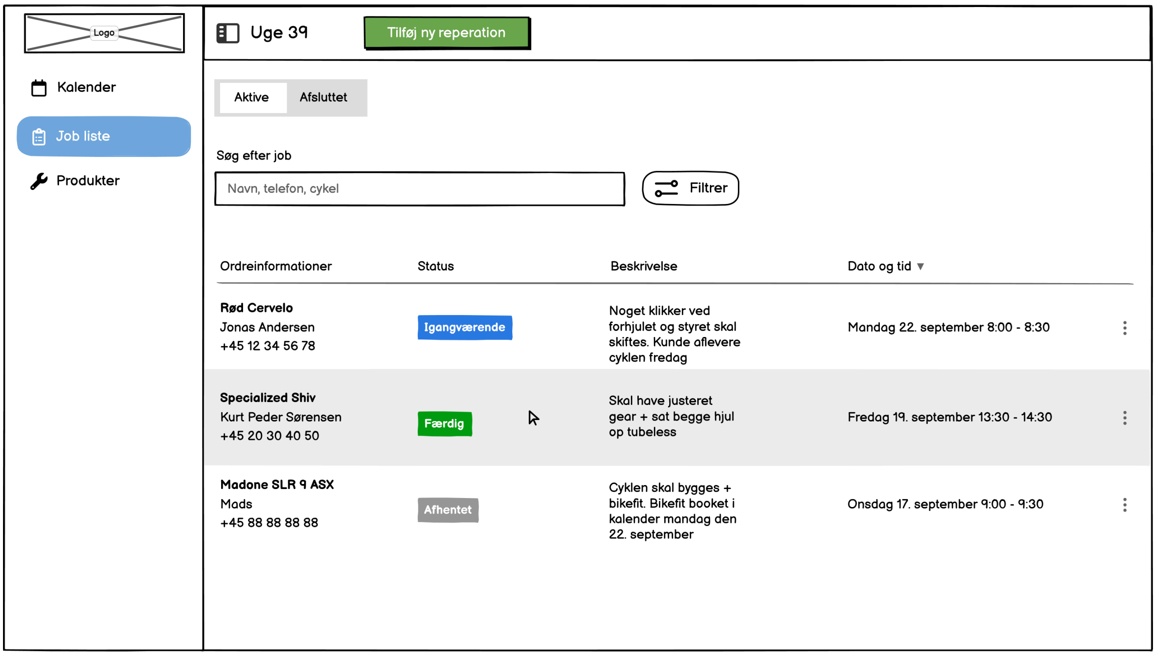


Figure 10: Lo-Fi Job List Wireframe

5.1.3 Lo-Fi Products

Figure 11 presents the initial low-fidelity wireframe for the *Product Page*.

The main content area is split into two functional parts. At the top, a search and add product panel enables users to create a new product by clicking the blue "Add Product" button or to locate existing items using a search bar that accepts a product's name, type, or ID. A filter icon allows users to further narrow the list, letting them display only spare parts or only services.

Below this panel, a structured product table lists all available items. Each row displays the product ID, name, type, and price, providing staff with a clear overview of the inventory. This page serves as the foundation for managing spare parts and service offerings within the system.

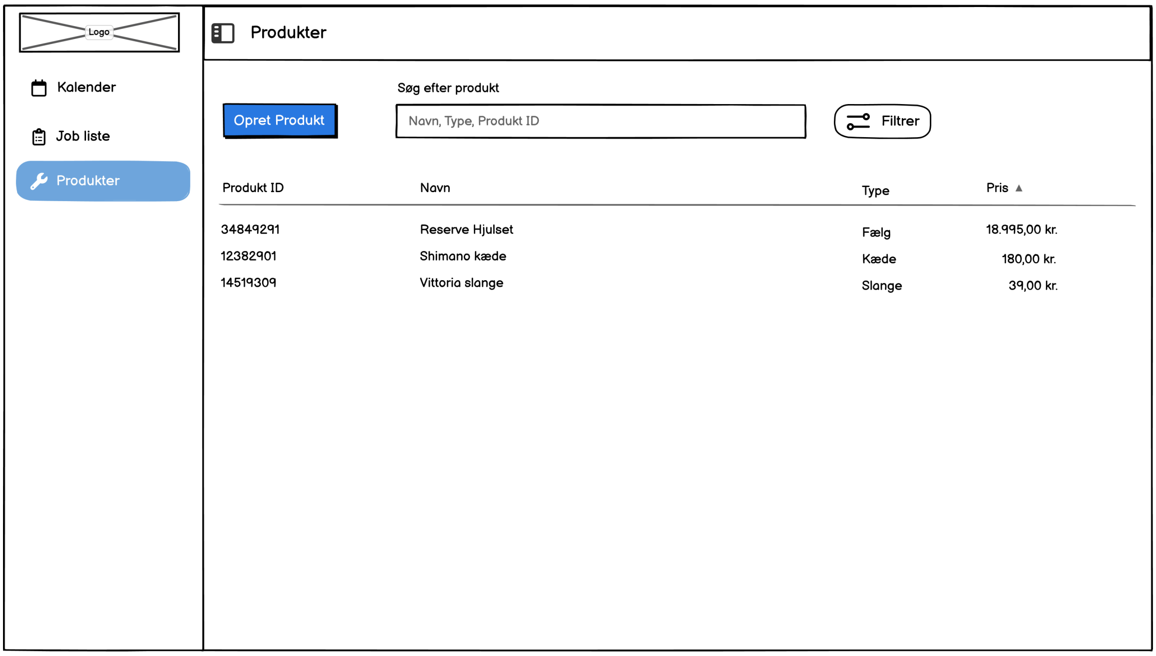


Figure 11: Lo-Fi Product Page Wireframe

5.1.4 Evaluation of Lo-Fi Wireframes

The above Lo-fi wireframes were evaluated by Performsport to ensure that they followed the preferred workflow and displayed the required information at the right times. Based on this discussion, a few changes were made from the Lo-fi to the Hi-fi wireframes.

The overall structure and information architecture of the pages remained the same, but "job liste" is now referred to as "reparationer" to match the terminology used by Performsport. Each Part now includes an EAN number, which is also reflected later in the Section 6.2.5, and the buttons on the calendar page were rearranged to improve the user experience.

In addition, a feature for adding services directly within the scheduled repair form was discussed. This allows Performsport to quickly calculate the cost and duration of a repair while speaking with a customer over the phone or in person at their physical location. This feature is shown in the Hi-Fi calendar wireframe Section 5.2.1.

5.2 Hi-Fi Wireframe

Hi-fi wireframes, much like lo-fi wireframes, illustrate the user interface of the product, but in a much more detailed way. Hi-fi wireframes are designed to resemble the final product. It is useful for a detailed evaluation of the main design elements in the final stages of client acceptance, as a final design document that needs to be agreed upon by the client, before implementation can commence [5].

5.2.1 Hi-Fi Calendar

Figure 12 represents the final design of the calendar interface, where all scheduled repairs are visible. The view of the calendar can be switched between day, week, and month, depending on preferences and possibly the need for a more or less detailed view. Each repair in the calendar has a special color that indicates the status of the repair (received, ongoing, finished, etc.), which can be changed by the employees. The left-hand side of the page contains a navigation bar for navigating the different pages of the website.

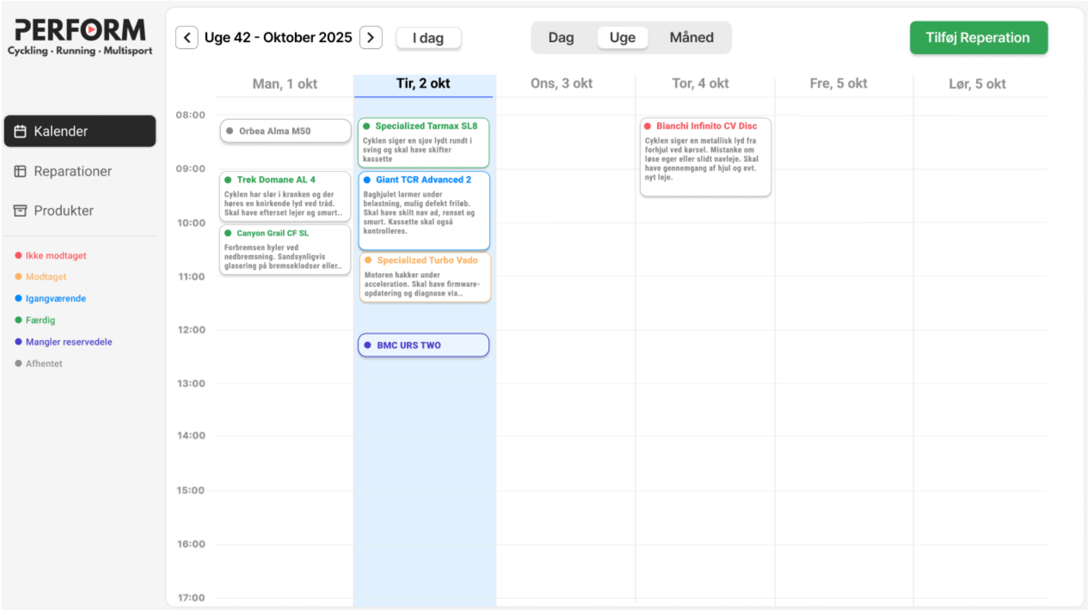


Figure 12: Hi-Fi Calendar Week-View Wireframe

Figure 13 represents the modal, or the pop-up window, that appears when "Tilføj Reperation" is pressed on the calendar page, which gives the option to create a new repair. The modal prompts the user to enter information about the repair (customer name and phone number, date of the repair, and services, etc.). When the necessary information is submitted, the repair will become visible in the calendar, as well as the repair page illustrated in Figure 14.

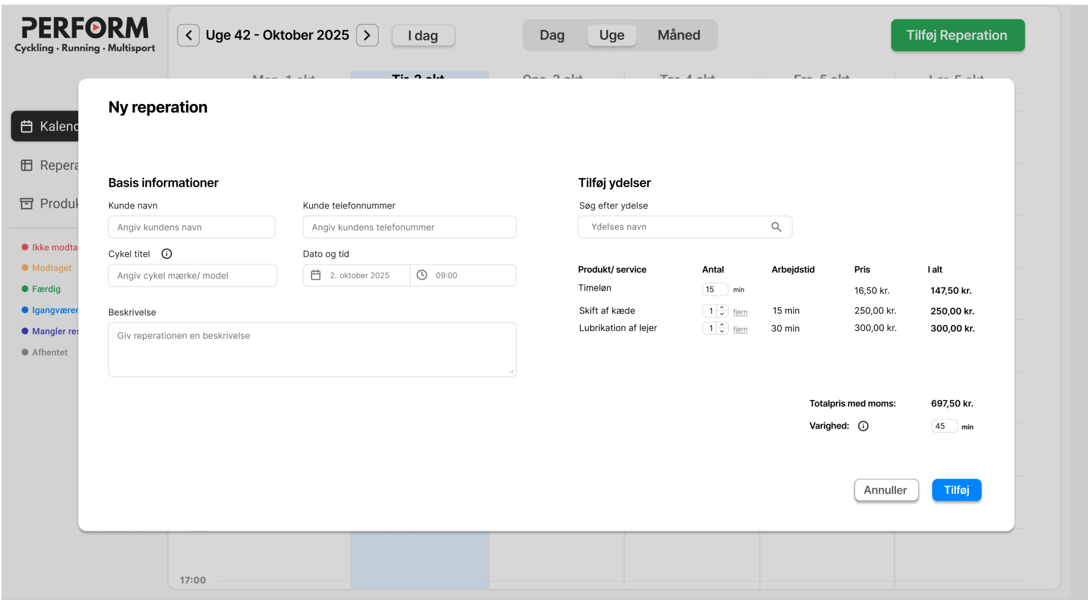


Figure 13: Hi-Fi "Add Repair" Form

5.2.2 Hi-Fi Repair List

Figure 14 illustrates the repair list, in which all scheduled and finished repairs are visible. The user has the option to search for specific repairs, using certain keywords or filters. This is especially useful in contrast to searching through each day of the calendar to locate a specific repair. A new repair can also be created directly on the repair list page by pressing "Tilføj

Reperation" in the top right corner, prompting the user with the same modal as illustrated in Figure 13.

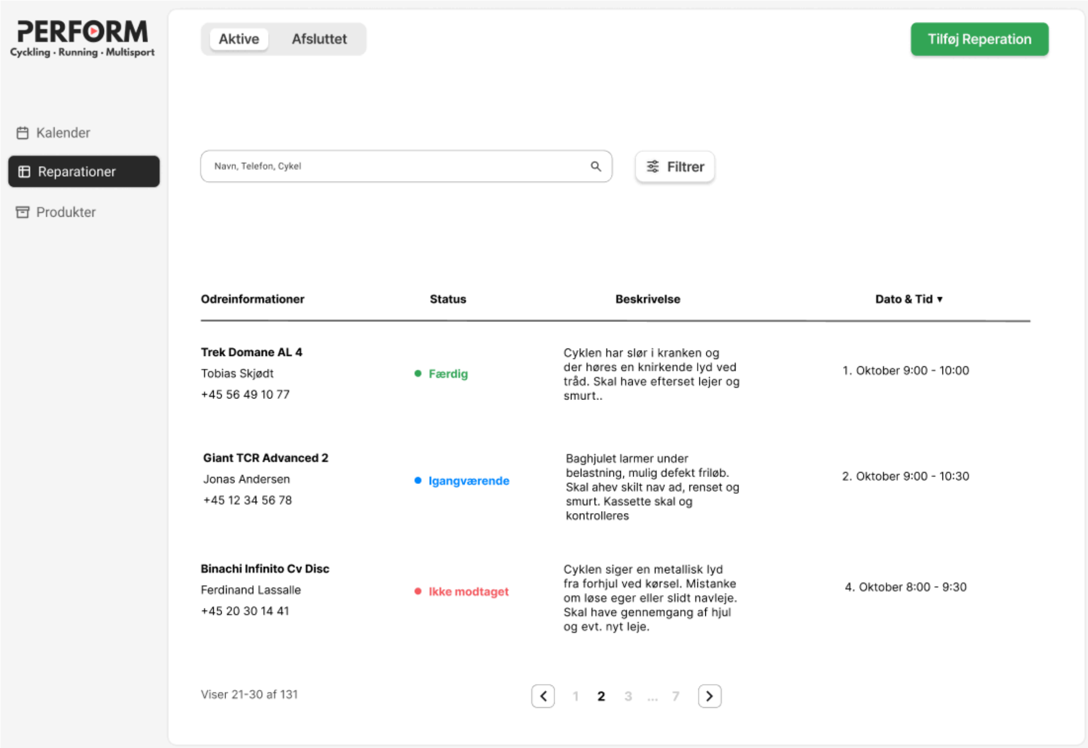


Figure 14: Hi-Fi Repair List Wireframe

5.2.3 Hi-Fi Products

Figure 15 is a list of all products and services offered by Performsport. This is especially useful when an employee needs to add a specific part or service to a repair job. Non-existing products or services can be added to the list by using the button in the top right corner "Nyt Produkt".

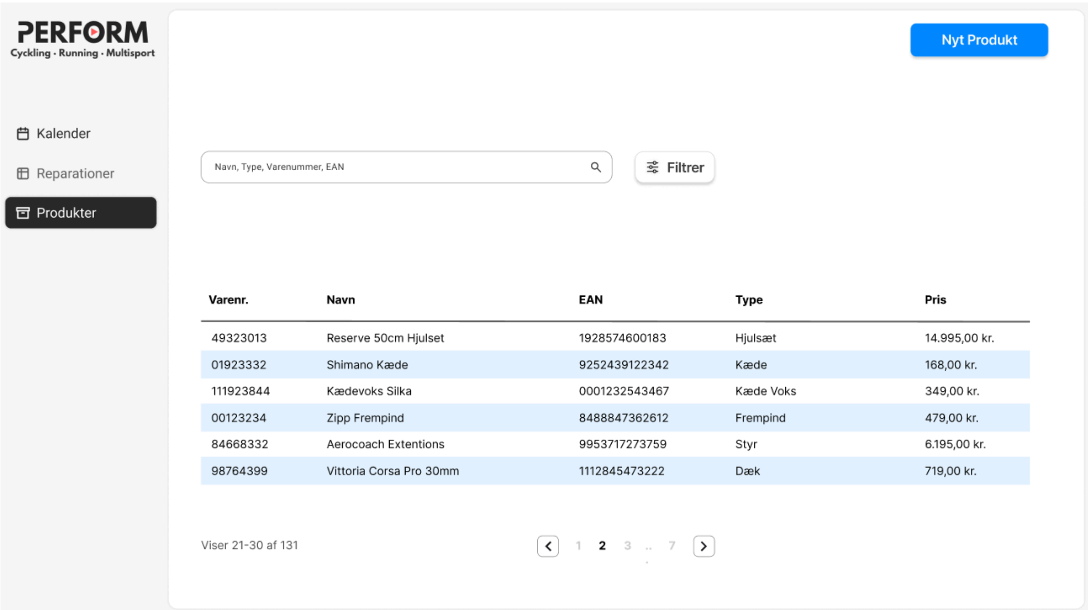


Figure 15: Hi-Fi Product List Wireframe

5.2.4 Evaluation of Hi-Fi Wireframes

The Hi-Fi user interface designs were presented to Performsport, who confirmed that the new features discussed during the Lo-Fi wireframing phase had been implemented in a way they could see themselves using. They also expressed satisfaction with the visual aesthetics of the design. With the user interface established, the architectural design of the application could then be developed, which is described in the next section.

6 Architectural Design

The architectural design defines the system's high-level blueprint, describing its criteria, structure, and processes. Design criteria establish the principles and constraints that guide decisions, the structure outlines the system's layers and components, and the processes explain how data flows through the system. Together, these elements ensure the system is efficient, maintainable, and aligned with its goals.

6.1 Design Criteria

In the first section of the architectural design, the classic design criteria from Object-Oriented Analysis are discussed and prioritized. This prioritization is based on the requirements defined using the MoSCoW method in Section 2.4 as well as insights gathered from the interview with Performsport.

Usable:
The system is designed to complement Performsport's existing bike repair workflow. Therefore, it must be easy to use and integrate seamlessly into the current process, rather than being perceived as an additional step. To achieve this, the system should resemble the existing workflow by prioritizing adaptation over strictly following the requirements set out in the analysis.

Secure:
In the system design, security has not been prioritized as a core concern, since the application is intended to run locally. As a result, measures such as authentication, encryption, and access control have not been implemented. This approach simplifies development, while still being appropriate for the project's context.

Efficient:
The system should optimize resource usage on the technical platform, minimizing factors such as consumption and processing overhead, and ensuring efficient database queries. A general, streamlined, and optimized system ensures a responsive program for employees.

Correct:
The system is implemented in accordance with requirements formulated for Performsport's specific use cases. Each implementation feature is mapped to a requirement made by Performsport. Unit tests and integration tests are employed to further ensure that the system meets the functional requirements and satisfies Performsport's needs correctly and consistently.

Reliable:
The system should execute functions accurately and consistently, ensuring integrity throughout all operations, such as creation, status updates, and pricing, among others. With reliability, it should uphold robust error handling and validation in case of data corruption and such.

Maintainable:
The system follows the structured object-oriented paradigm together with Spring Boot, making add-ons and maintainability possible for future corrections, minimizing disruption in existing functionality. This requires a clear code structure and a well-structured architecture.

Testable:
The system should be designed to be easy to test and validate. The system's modular architecture and clear separations enable comprehensive unit and integration testing.

Flexible:
To maintain flexibility in the design, the requirements are defined with consideration for potential future changes. Modularity plays a key role in this approach, involving the division of the system into smaller, independent modules. This structure makes the system easier to modify, extend, or upgrade in the future.

Comprehensible:
The system front-end structuring and architecture must be logically organized, minimizing the effort required to understand the different functionality in the system. This is largely achieved with good user interface design and by following typical UX conventions.

Reusable:
Components can be designed with modularity in mind, enabling their use in similar scheduling or service management systems, however, that is not a criterion in the case of this system.

Portable:
The system and especially the components, should be easily transferable from one environment to another without the need for significant modification. This ensures that the system works across different platforms, hardware, or environments

Interoperable:
As mentioned in Section 2.3, the calendar system is intended to be a standalone application and will not initially access information from Performsport's existing systems. However, in the MoSCoW prioritization, integration with the current Shopify order and inventory solutions was classified as a could-have requirement and may be implemented if the project scope allows. Regardless, the system should be designed with future Shopify integration in mind, ensuring that features such as creating a draft order from a repair description and managing inventory within the system can be added easily.

The importance of the above design criteria is ranked in Table 8. The most important design criteria for the system are usability and comprehensibility. Criteria such as security, portability, and interoperability are not ranked as important for the system. As the components will not be reused for future programs, the reusability is ranked as irrelevant.

Criterion	Very Important	Important	Less Important	Irrelevant
Usable	X			
Secure			X	
Efficient		X		
Correct		X		
Reliable		X		
Maintainable			X	
Testable		X		
Flexible			X	
Comprehensible	X			
Reusable				X
Portable			X	
Interoperable			X	

Table 8: Ranking of Design Criteria Based on Individual Importance to the System

The system fulfills these requirements through a layered Spring Boot architecture designed for modularity and maintainability. Usability is achieved by closely mirroring Performsport's existing workflow in the application, preserving familiar processes as much as possible. Correctness and reliability are ensured through extensive unit and integration testing. The Model-View-Controller (MVC) architecture enhances overall testability by enabling clear separation of concerns, easy isolation, and the reuse of generic components for flexibility. Finally, comprehensive documentation and consistent coding practices ensure clarity and ease of understanding for future development.

The next section describes the components of the system and the overall architecture of how these components work together, with the interface, controller, service, model, and technical layer. These are designed with the design criteria in mind.

6.2 Component Architecture

The purpose of visualizing the component architecture is to define the detailed structure and responsibilities of each element within the system. The Performsport calendar system in Figure 16 is organized into five layers: the interface layer, which consists of the user interface and system interface, the controller layer, which includes the page controller and data controllers, the service layer, which provides functions to manage the calendar and scheduled repairs, the model layer, which represents the real-world objects of the problem domain, and the technical layer, which comprises of the database and Spring Boot, an open-source framework for Java web development.

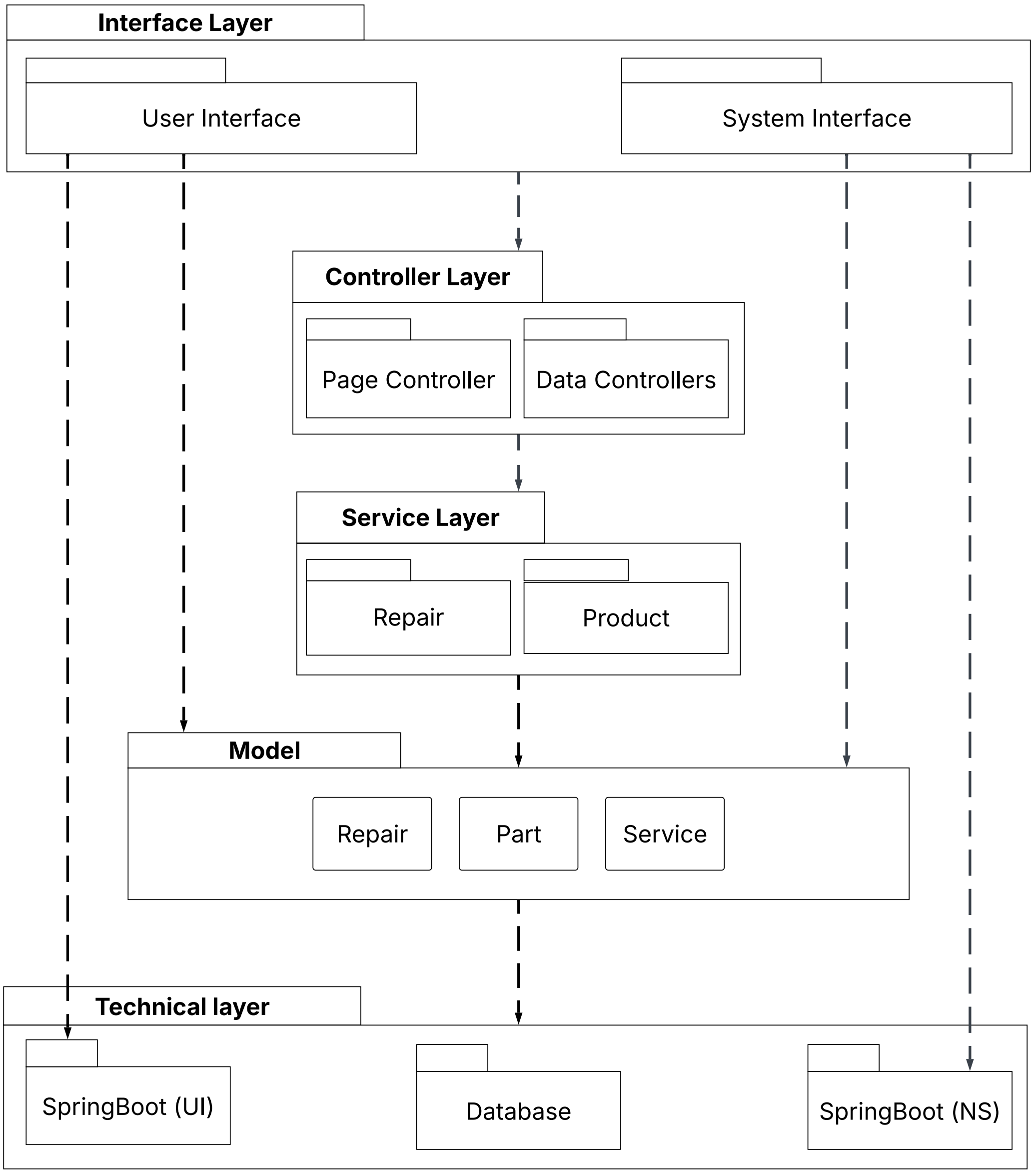


Figure 16: The Architectural Structure of the Application as a Component Diagram

Each component in Figure 16 is described in more detail in the sections below, starting with the interface layer.

6.2.1 Interface layer

The interface layer consists of a user interface component (UI) and a system interface component. The user interface component provides the point of interaction between the actors and the system's functionality, enabling them to perform the use cases described in Section 4.1, such as creating repairs or providing information for an invoice. The system interface defines how the application communicates with external systems and services, such as the database and the APIs in the controller layer.

In the component diagram, the interface layer also connects directly to the technical layer. This is because the interface uses technologies, such as Thymeleaf [6] for dynamically rendering HTML content, and the network system (NS) in Spring Boot for handling API requests and hosting the application locally.

6.2.2 Controller Layer

The controller layer connects the user interface with the application's logic and handles user inputs and system requests. It is divided into two main components: a page controller for navigation and data controllers for handling data exchange between the front-end and back-end. This structure separates navigation from logic and data handling, improving flexibility. With this separation, a front-end library such as React or Vue could be implemented later without changing the back-end logic.

Page Controller: Handles navigation between different views, such as the calendar, repair list, and product pages. It ensures that the correct HTML templates are displayed to the user at the appropriate times. The system uses Thymeleaf, a server-side Java template engine integrated with Spring MVC, to generate dynamic HTML pages. Thymeleaf allows data from the application model to be inserted directly into HTML templates using simple expressions, enabling seamless integration between back-end logic and front-end presentation.

Data Controllers: The data controllers manage the exchange of data between the front-end and the back-end. This is done using a REST architecture, implemented through the *@RestController* annotation in Spring. Each method in a data controller corresponds to an HTTP operation, such as GET, POST, PUT, or DELETE. The two main controllers in the system are the *RepairController*, which handles creating, editing, and searching for repairs, and the *ProductController*, which manages products used in repairs. When a client sends a request, such as creating a repair, the controller forwards it to the relevant service in the service layer. This separation improves maintainability by clearly distinguishing between display logic and data processing.

6.2.3 Service Layer

The service layer contains the business logic and implements the core functionality of the application, like scheduling repair and adding new products to a repair. It coordinates interactions between the controller and model layers. The two primary services are:

Repair Service: The *RepairService* is responsible for managing all business logic related to repair jobs. This includes creating, updating, and deleting repairs, as well as handling their associated data such as status, assigned date, and total price. It coordinates with the *ProductService* to manage the parts and services linked to each repair and ensures that calculations for price and duration are consistent with the products used. The service also enforces business rules, such as ensuring that the customer is charged for labor costs and setting the duration of a repair based on the complexity of the included services.

Product Service: The *ProductService* manages the operations related to products, which include both parts and services. Its responsibilities cover creating, updating, deleting, and retrieving product information, as well as supporting search and filtering operations used on the inventory page. It ensures data consistency across the system and provides product data to other services, such as the *RepairService*, when repairs require products to be added.

The service layer calls the repository layer to store and retrieve information from the database.

6.2.4 Repository Layer

The Repository Layer is a technical sub-layer responsible for managing all communication between the application and the database. It provides an abstraction, allowing the rest of the system to interact with the database through simple methods rather than using database queries. Each repository corresponds to a model class, such as Repair or Product, and handles the basic CRUD operations, Create, Read, Update, and Delete, for that entity.

In this program, each repository is responsible for retrieving and managing data from the database for its corresponding model class. Repositories are used, for example, to find a repair by its specific ID or to retrieve all products associated with a particular repair.

6.2.5 Model Layer

The Model Layer represents the problem domain and encapsulates the entities required for the bike repair system. Each class corresponds to a real-world object in Performsport's environment:

- *Repair*: Contains repair information, customer details, and status.
- *Part*: Represents spare parts, including cost and quantity.
- *Service*: Represents predefined services with associated duration and price.

The model layer ensures consistency in how object-related data is stored and manipulated throughout the system. Objects are persisted in a database, where each column corresponds to an attribute and each row represents an individual object. When relevant events occur in the system, such as a service price update or a spare part being added to a repair. The model layer reflects these changes through updates to the corresponding entities. This layer, therefore, forms the foundation for the repository and service layers, which use the model classes to manage and process data.

Figure 17 illustrates the class structure of the model component, showing the relationships and data contents of each class.

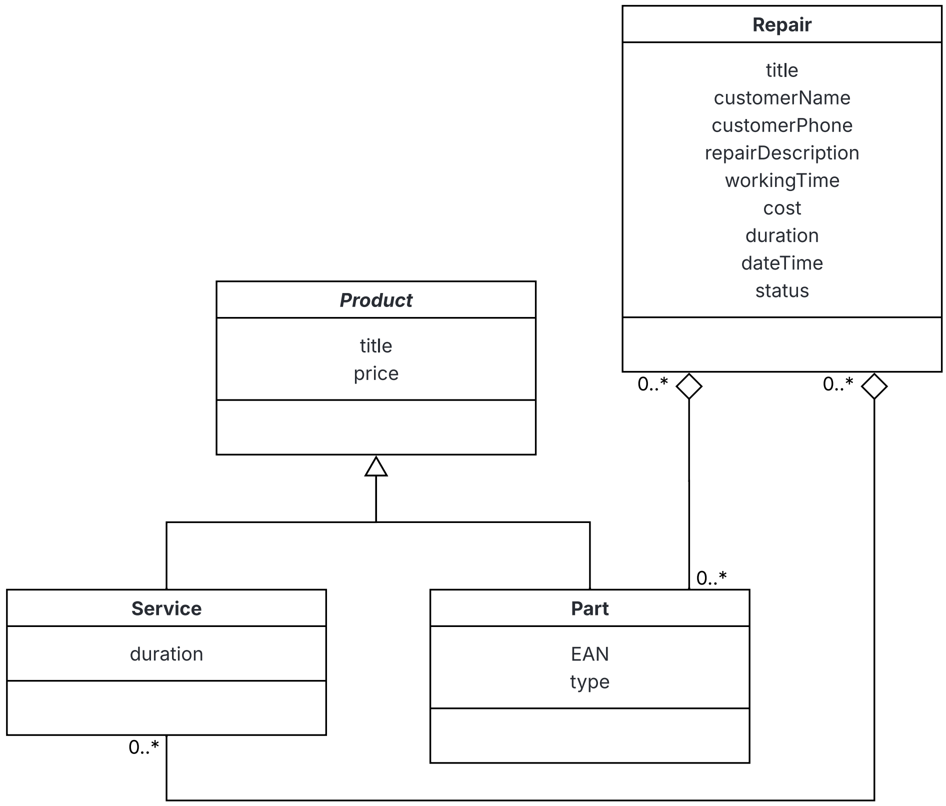


Figure 17: Revised Class Diagram from Figure 2 with Attributes

Figure 17 shows an updated version of the class-diagram from Section 3.4 with attributes for each class. The repair class is the main class of the model layer, which holds information about all the repairs at Performsport. Each repair will have a title, information about the customer, a bike description, work time (used to calculate labor cost), total cost of the repair, a duration (used in the calendar), a date, and finally a status.

The other objects in the system are the parts and services. These are modeled through a hierarchical structure, where the Part and Service class inherit a title and a price attribute from the abstract product class. In addition to this, the each part will have an EAN number and a type, and each service will have a duration.

Repairs are associated with products through aggregation, as explained in Section 3.4. The program could use a link-table (or join-table) to resolve the many-to-many relation between repairs and products.

An example of how a repair is represented in the model layer, is shown in the object diagram in Figure 18. The diagram shows a repair for a *Blue cervelo* bike, with two associated parts and one associated service. The price and duration of the repair, is calculated using the attribute values of the parts and services, as seen in the diagram.

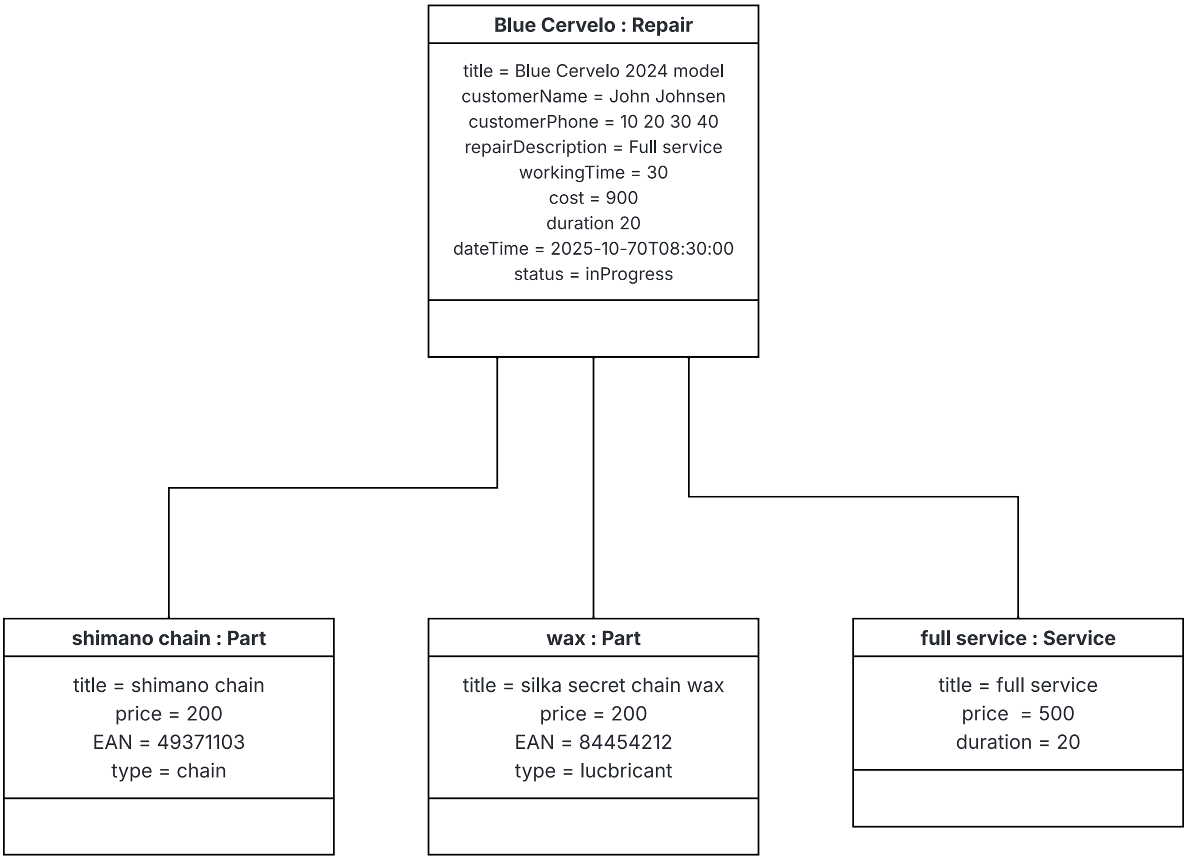


Figure 18: Illustration of an Example Repair Instance in the Model Layer as an Object Diagram

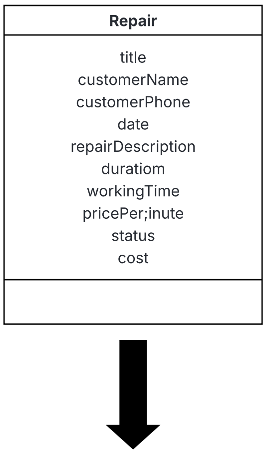
The next section describes the technical layer of the program, which defines the underlying technologies and frameworks, that support the functionality of the system.

6.2.6 Technical Layer

The technical layer provides the foundation upon which the architectural layers operate, handling aspects such as data storage, communication between components, and integration with external systems. In this program, the technical layer, or tech-stack, includes the Spring Boot framework with Thymeleaf for application structure and dependency management, Spring Data JPA for database interaction, and a PostgreSQL database for storage. The front-end is written mainly in plain HTML and Javascript, but uses Bootstrap [7], an open source front-end framework, to make front-end development faster, in order to focus on more important parts of the application. Together, these technologies ensure efficient communication between layers, maintain data integrity, and support a modular and maintainable system design.

Spring Boot: Spring Boot was chosen for this web application because it provides an efficient, modular, and production-ready framework for backend development. Its auto-configuration and dependency management reduce setup complexity and enable faster implementation of core functionality [8]. Spring Boot also integrates seamlessly with relational databases through Spring Data JPA, simplifying data persistence and CRUD (Create, Read, Update, Delete) operations [9]. Overall, Spring Boot offers a robust and scalable foundation that supports quick development, maintainability, and reliable data management, making it a suitable framework for this project. Thymeleaf is integrated with SpringBoot and is used to create static HTML templates with dynamic content.

Database: A PostgreSQL database stores all the constant data, including repairs, customers, services, and parts. Each component in the model layer maps to a corresponding table, ensuring data integrity. Figure 19 presents an example of how the Repair class-object could be transformed into a table in the database:



id	title	customer_name	Customer_phone	date	job_description	duration	working_time	price_per_minute	status_id	cost
1	Red Cervelo Bike	Hans Andersen	01234567	2025-10-08T15:30:00	The bike needs a new chain and bearings has to lubricated	40	60	8.5	3	1000
2	Blue Specialized Bike	Birgitte Madsen	44334422	2025-10-09T14:00:00	Full service + new handlebars	120	120	8.5	2	2400

Figure 19: Visualization of how Repair Objects are in a Data-Table

Docker: Docker is an open platform technology that allows application and their dependencies to be packaged into portable containers, also known as containerization. Containerization allows software to be packaged together with all the components it needs, such as libraries, system tools, and configuration files, into a single entity called a container [10].

A container works as an isolated environment that runs an application consistently across different systems, which helps avoid issues where it "works on my machine, but not on yours." Additionally, containers do not require their own operating system for each instance, they share the host system's resources, making them more efficient and faster to start [10].

In the context of this project, Docker is used to host the database inside a container. Instead of installing the database software directly on the host system, Docker runs it from a database image, which is a predefined package/library that contains everything needed to create the database environment (such as the database engine, configuration settings, and dependencies). When a container is started from this image, it creates a fully functional instance of the database that is isolated from the rest of the system [10].

Docker integrates into the overall system architecture by providing the infrastructure layer for the database. While the Spring Boot application manages data access through its Repository and Service layers, as seen in Section 6.2, Docker hosts the PostgreSQL database in an isolated container environment. This separation ensures that the application and database remain independent, but fully connected via a controlled network, which improves maintainability and provides consistency in the system.

Overall, Docker provides a reliable and reproducible environment for the database in this project, improving both development efficiency and system stability. While portability is not a primary design criterion for the application itself, Docker contributes to portability at the infrastructure and development level by ensuring consistent database environments across systems.

This approach offers several advantages:

- *Consistency*: The same database image can be used across all development and testing environments, ensuring identical configurations.
- *Isolation*: The database container operates independently from the application, preventing configuration conflicts or version mismatches.
- *Portability*: Containers can easily be moved between computers or servers without reinstallation.
- *Simple Setup*: The database can be started or stopped using a single Docker command, reducing setup time.

Docker Architecture
Docker follows a client-server architecture that manages the building, distribution, and execution of containers.

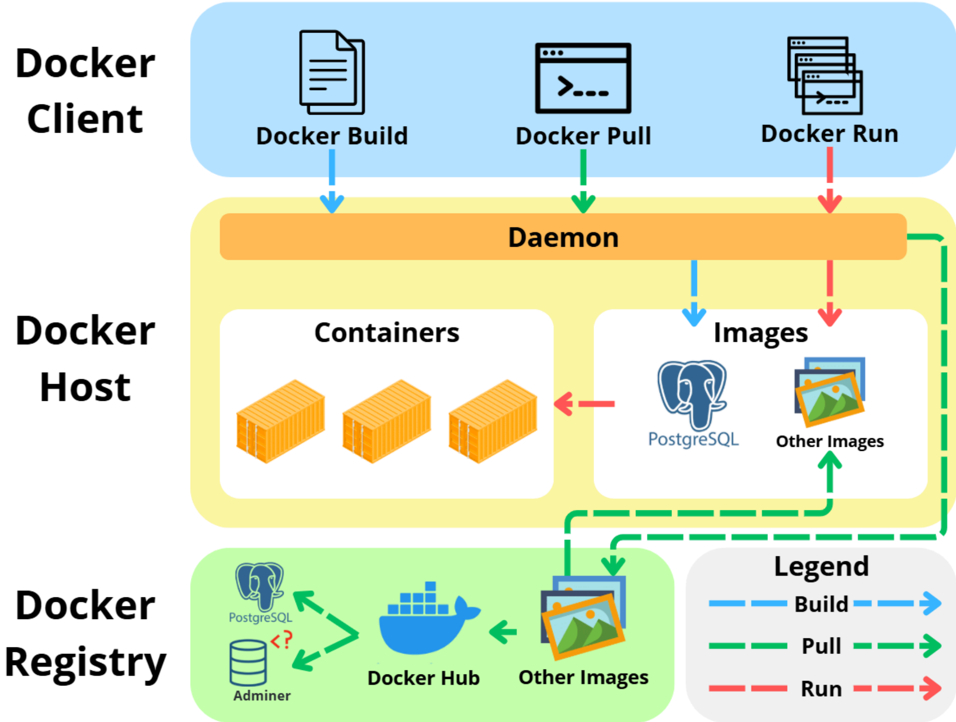


Figure 20: Visualization of Docker Architecture from [10]

The main components are [10]:

- *Docker Client*: Sends user commands (e.g., `docker run`) to the Docker daemon.
- *Docker Daemon*: The background service that builds, runs, and manages containers and images.
- *Docker Images*: Read-only templates that define what a container includes, such as PostgreSQL or Adminer setup.
- *Docker Containers*: Runnable instances of images that operate in an isolated environment while sharing the host's operating system.
- *Docker Registry*: Stores and distributes images, such as those retrieved from Docker Hub (a registry/collection of downloadable images).
- *Volumes & Networks*: Provide constant data storage and communication between containers.

When *docker-compose up* is executed, Docker pulls the required images, creates containers, connects them via a network, and starts both services. This architecture enables an efficient, isolated, and portable database system, ensuring that the database environment remains identical across all systems.

This section outlined the system's overall architectural structure, describing how its layers, technologies, and design criteria form the foundation of the solution. It serves as a blueprint that guides development and establishes the framework for implementation. The following section builds directly on this design by translating the described architecture into concrete code and functionality.

7 Implementation

This section describes the implementation of the functionality described in the application domain section and the components illustrated in Section 6.2. The section is split into two parts: back-end implementation and front-end implementation. This section also includes a description of changes made to the application architecture, earlier described in Section 6.2.

The implementation will end with a concrete example of how one of the key use-cases from Section 4.1 is implemented in the application. In this case, adding a product to a repair.

Only key functions and code snippets are featured here. The full code-base is attached to the project and can also be found on the GitHub referenced in the Preface.

7.1 Architectural changes

Section 6.2 describes the architecture of the application. This architecture includes an interface layer, a controller layer, a service layer, a model layer, and a technical layer, as seen in the Figure 16. Although this architecture would have worked well, a repository layer was added during the implementation to make interactions between the application and the data storage system easier to handle and to separate data access logic from business logic.

A revised version of the component diagram from Section 6.2 in Figure 21 includes a repository layer between the model layer and the database. This repository layer contains a set of classes that implement reading and writing data between the domain model objects and the database. Using a repository pattern helps minimize duplicate database queries and also decouples the models from the database layer, so database tables can be changed in the future without having to change other parts of the program. The implementation of the repository layer is described in more detail in Section 7.2.4.

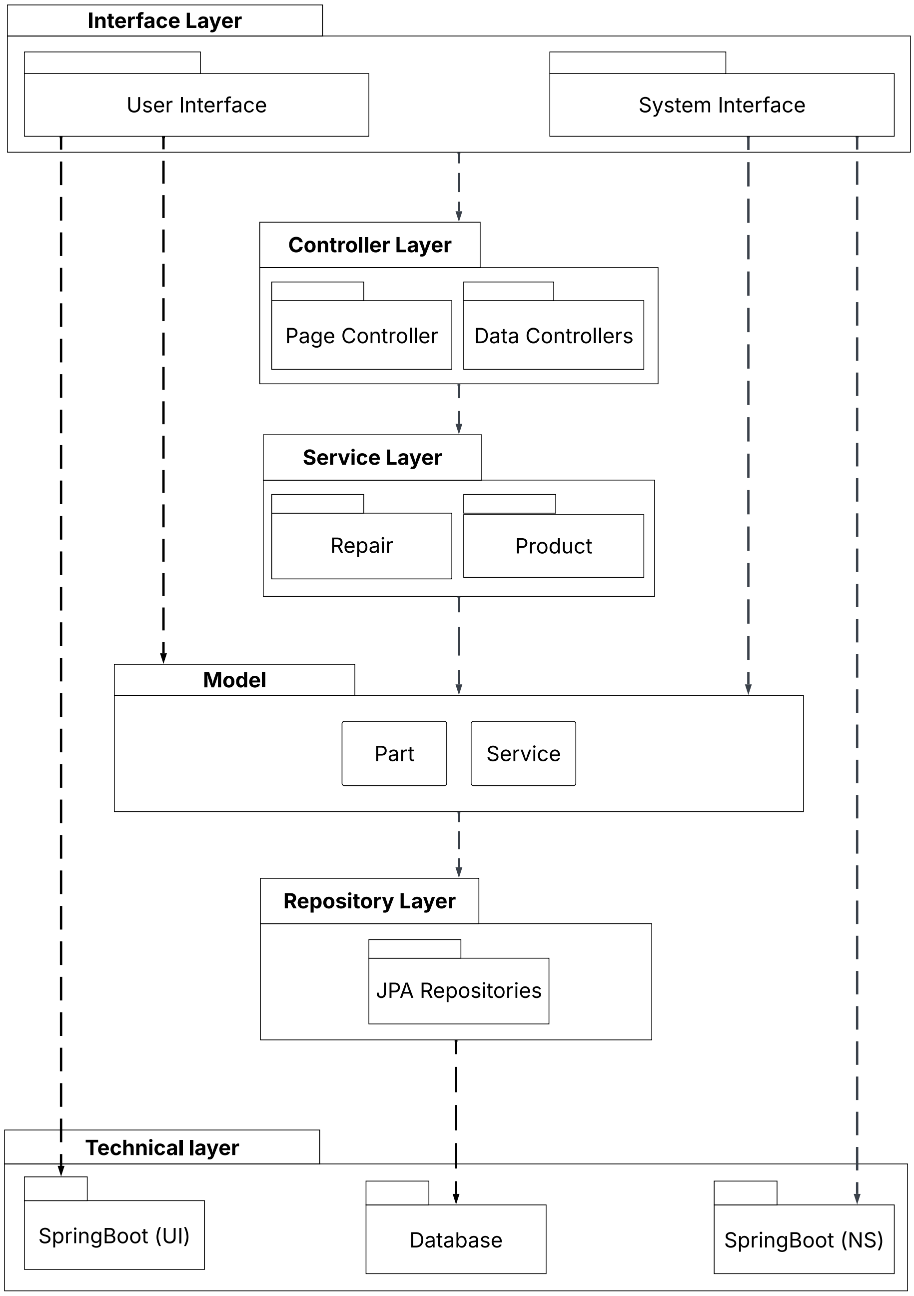


Figure 21: Revised Component Diagram from Figure 16 with the Addition of a Repository layer

During implementation, the model layer diverged from the original component architecture. In the initial design, the model contained the classes Product, Part, and Service, where Product acted as a superclass providing shared fields such as title and price.

However, in the implemented system, parts and services are never used polymorphically. Because the superclass does not provide functional value and only increases structural complexity, it was removed from the implementation. Therefore, Product is not included in the component diagram shown in Figure 21.

The application could be rewritten to use the Product superclass, e.g., when showing both parts and products on a list or when calculating a total price including both parts and services, however, this was left outside of this version of the program. A detailed description of the revised domain model is provided in Section 7.2.1.

7.2 Back-End

The first part of the implementation section describes the implementation of the system's back-end. It includes the implementation of the various components shown in the component diagram in Figure 16, including the model component, the API endpoints in the controller components, the service components that handle the application's business logic, and finally, the implementation of the database and data access.

7.2.1 Domain Model

During the early stages of implementation, the classes representing the application domain were named Job, Product, and Services. These names were introduced before a vocabulary had been established, together with Performsport, resulting in a mismatch between the domain terminology and the implementation. In the current version:

- **Job** represents a repair task
- **Product** represents a part
- **Services** represents a service

Note: the name Services is used instead of Service to avoid conflicts with the Spring @Service annotation.

Although these names are not fully aligned with the problem domain, the functionality of the application is not affected, and the terminology was not changed during development to avoid unnecessary refactoring. However, a future version of the system could rename the classes to improve readability and maintainability.

Using JPA entities

The domain model is implemented using JPA entities. An entity is a regular Java class annotated with `@Entity`, indicating that it should be persisted to the database. Each entity corresponds to a table, and each instance of the entity represents a row in that table.

The `@Table` annotation is used when the database table name should differ from the class name. Primary keys are defined using `@Id`, and in this application, all entities use `@GeneratedValue(strategy = GenerationType.IDENTITY)`. In PostgreSQL, this maps to an auto-incrementing identity column.

An example of the Job entity is shown below:

```
@Entity
@Table(name = "jobs")
public class Job {
    // Auto generated id for database management
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer ID;
    private String title;
    private String customer_name;
    private String customer_phone;
    private String job_description;
    private Integer work_time_minutes;
    private Double price_per_minute;
    private Integer duration;
    private LocalDateTime date;
    ...
```

```
    // Getters & Setters
    ...
}
```

The other domain classes, Part and Services, are implemented in the same way. Their relationships to Job (e.g., how parts and services are added to a repair) are described in the following sections.

7.2.2 API Endpoints

The Controller Layer is responsible for handling all incoming HTTP requests from the client and responding appropriately. In this project, controllers fall into two categories based on their purpose:

1. *Page Controller*: Responsible for returning HTML views rendered with Thymeleaf.
2. *REST Controllers*: Responsible for handling API operations such as creating, updating, and deleting jobs, parts, and services.

This separation ensures a clean architecture where the Page Controller handles navigation and view rendering, and API Controllers handle data operations and business logic delegation.

A controller acts as the bridge between the user interface and the base business logic. It does not contain the logic itself, but delegates work to the appropriate service classes. The controller's job is to: receive and validate HTTP requests, map requests to the correct service method, convert service results into HTTP responses, and handle errors, returning status codes accordingly. This separation ensures a clean structure where controllers manage communication, services manage logic, and repositories handle persistence, meaning data is saved and retrieved from the database so it remains available across usage.

Figure 22 illustrates the general flow of a controller-based API, showing how requests move from the client through the controllers and service layer before reaching the database.

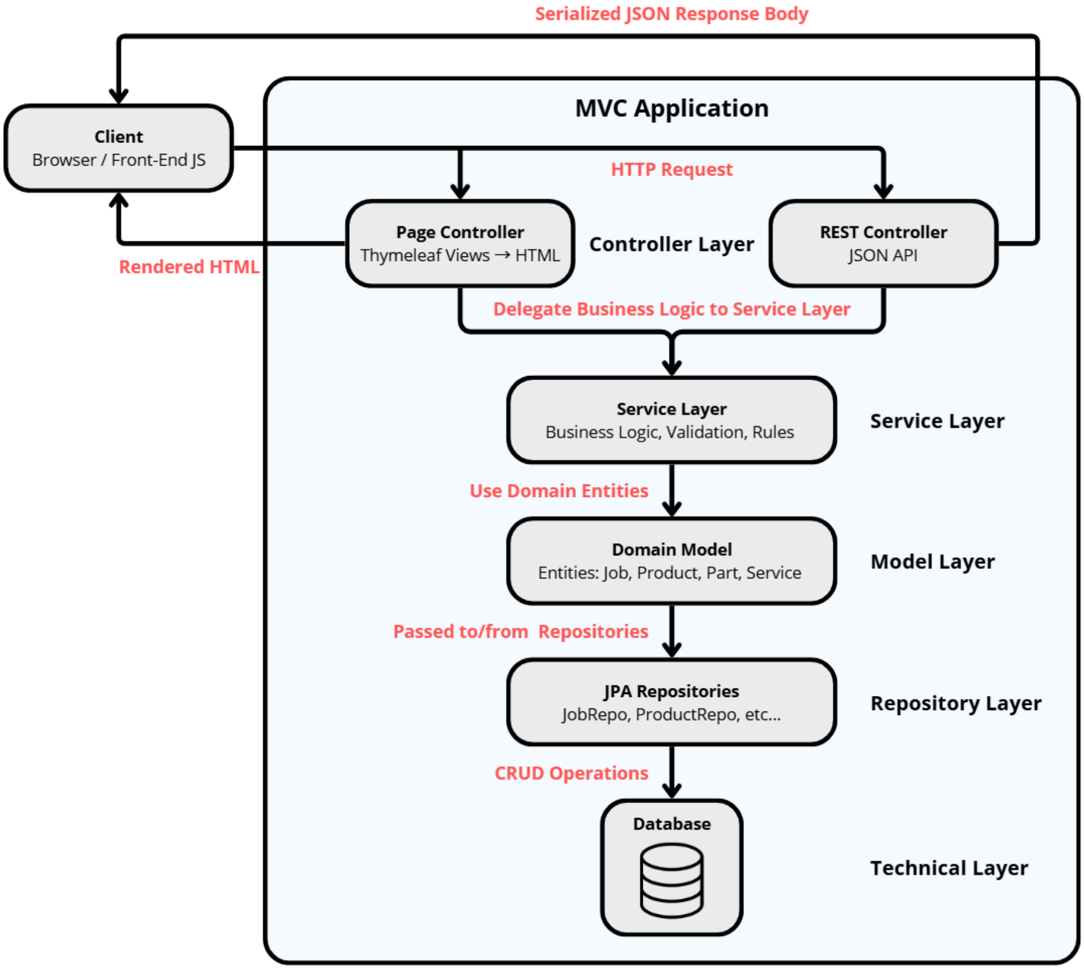


Figure 22: Illustration of a Controller-Based Web API Structure

The Structure of Controller Endpoints

Because Page Controllers and REST Controllers serve different purposes, they follow different structural patterns.

A **View Controller** always responds to `GET` requests, since its only role is to display pages and never modify data. When a request comes in, the controller may optionally prepare model data by retrieving information from repositories or services and adding it to a `Model` object. Finally, it returns the name of a Thymeleaf view, which Spring resolves to the corresponding HTML template. This makes View Controllers simple and focused entirely on navigation and rendering user interfaces.

An **API Controller**, on the other hand, follows the typical structure of a REST endpoint. It uses HTTP methods such as `GET`, `POST`, `PUT`, or `DELETE`, depending on the type of operation being performed. Instead of preparing views, an API Controller delegates the actual work to a service method, which contains the business logic for creating, updating, or deleting data. Once the service finishes executing, the controller returns an HTTP response, often JSON data or a status code, wrapped in a `ResponseEntity`. This keeps API controllers compact, consistent, and aligned with REST principles.

View Controller Example - PageController:
The application utilizes a single main `PageController` for navigation and rendering Thymeleaf pages, including the calendar, job list, job details, and product view.

```
/* --- PageController Class --- */
// Controller responsible for serving HTML pages (Thymeleaf templates).
// Handles navigation, prepares model data, and returns the view names.
// Unlike REST controllers, this returns the template name (resolved to HTML)
// rather than JSON.
@Controller
@RequestMapping("/")
public class PageController {
    // Handles requests to the root URL and redirects to the calendar page.
    @GetMapping("")
    public String home() {
        return "redirect:kalender";
    }

    // Displays the calendar page showing all jobs in a calendar view.
    @GetMapping("kalender")
    public String calendarPage() {
        return "calendar";
    }

    // Displays the job list page with all jobs sorted by date in ascending order.
    // Populates the model with a list of all jobs for rendering in the template.
    @GetMapping("jobliste")
    public String jobList(Model model) {
        model.addAttribute("jobs", jobRepository.findAllByOrderByDateAsc());
        return "jobliste";
    }

    // Displays detailed information for a specific job, including associated parts.
    // Retrieves the job and its related parts, then populates the model for
    // the detail view.
    @GetMapping("jobliste/{id}")
    public String jobDetails(@PathVariable int id, Model model) {
        // Fetch job by ID using the service layer
        Job job = jobService.getJobById(id);
        model.addAttribute("job", job);

        // Fetch all parts associated with this job from the repository
        model.addAttribute("jobParts", jobPartRepository.findByJobId(id));

        // Fetch all service associated with this job from the repository
        model.addAttribute("jobService", jobServiceRepository.findByJobId(id));

        // Return the view template for rendering ("jobDetails.html")
        return "jobDetails";
    }
}
```

This controller only provides views and prepares model data for the templates. No data modification happens here.

API Controller Example - JobController:
The `JobController` includes REST endpoints used by the front-end JavaScript when creating, updating, or deleting repairs. These endpoints return JSON rather than HTML.

```
/* --- JobController --- */
// REST Controller responsible for handling all incoming HTTP requests
// related to Jobs/Repairs.
// The Controller does NOT contain Business Logic
@RestController
@RequestMapping("/")
public class JobController {
    // Get All Jobs - GET
    /** @return List of all Job records in the database **/
    @GetMapping("api/jobs")
    public List<Job> getJobs() {
        // Delegate the request to the Service Layer
        return jobService.getAllJobs();
    }

    // Create a New Job - POST
    /** @param body a map containing all fields needed to create a Job **/
    /** @return ResponseEntity containing the created Job or an error message **/
    @PostMapping("/api/jobs/create")
    public ResponseEntity<Job> createJob(@RequestBody Map<String, Object> body) {
        // Forward creation logic to the Service Layer
        return jobService.createJob(body);
    }

    // Update a Job - UPDATE
    /** @param id the ID of the job to update **/
    /** @param job the update job object sent from the front-end **/
    /** @return ResponseEntity with the updated job or a not found error **/
    @PutMapping("api/jobs/{id}/update")
    public ResponseEntity<Job> updateJob(@PathVariable Integer id,
                                         @RequestBody Job job) {
        // Service layer handles validation and updating
        return jobService.updateJob(id, job);
    }

    // Delete a Job - DELETE
    // Marked as @Transactional to ensure consistency if multiple deletes occur.
    /** @param id ID of the job to delete **/
    /** @return HTTP 204 No Content if successful or 404 if not found **/
    @DeleteMapping("/api/jobs/{id}")
    @Transactional
    public ResponseEntity<Void> deleteJob(@PathVariable Integer id) {
        // Pass the delete request to the Service Layer, which validates the job ID
        // and performs the deletion. If the job does not exist, a 404
        // response is returned.
        return jobService.deleteJob(id);
    }
}
```

These API endpoints are primarily used by the calendar interface and the job detail page to dynamically manage repair data.

7.2.3 Service Layer Implementation

The service layer contains all the business logic of the application. As described in the section above, the controller layer uses service classes to handle functions like creating a repair, adding products to a repair, or creating and editing parts and services.
The implementation of the application uses a *JobSerivce*, *ProductService*, *ServicesService*, and also a *BaseSearchService* interface, which the other services implement and override with their own search method. The @Service annotation is used in the implementation to tell Spring Boot that the class is a service component.

The *JobService* component is responsible for creating, updating, and deleting jobs, as well as managing the products and services associated with a repair. The code snippet below demonstrates the method used to add a service to an existing repair. This method accepts the ID of the repair, the ID of the service to be added, and the quantity of that service. For example, if a repair involves replacing both the front and rear wheel bearings, the quantity would be set to two.

The method begins by locating the repair using the provided repair ID. This is done through another method within *JobService*, which queries the *JobRepository* to retrieve the corresponding repair. It then retrieves the service to be added by querying the *ServiceRepository* directly.

Once both the repair and service have been successfully located, the method attempts to create or update the join-table entity *JobServices*, which represents the relationship between a repair and a service. Each *JobServices* instance stores the repair ID, the service ID, and the quantity, allowing the system to track which services are associated with which repairs (the functionality of this join-table is described in Section 6.2.4).

If a *JobServices* entry already exists for the given repair and service, the method increments its quantity by the amount specified in the request. If no such entry exists, a new *JobServices* instance is created and saved through the *jobServiceRepository*.

```java
// Add Service to Repair - HELPER METHOD
public void addServiceToRepair(int repairId, int serviceId, int quantity) {
    // Fetch the repair entity, throw the exception if not found
    Job repair = jobRepository.findById(repairId)
            .orElseThrow(() -> new RuntimeException("Repair not found"));

    // Fetch the service entity, throw an exception if not found
    Services service = serviceRepository.findById(serviceId)
            .orElseThrow(() -> new RuntimeException("Service not found"));

    // Check if service is already linked to this job
    JobServices existing = jobServiceRepository.findByJobId(repairId)
            .stream()
            .filter(x -> x.getService().getId() == serviceId)
            .findFirst()
            .orElse(null);

    if (existing != null) {
        existing.addQuantity(quantity);
        jobServiceRepository.save(existing);
    } else {
        jobServiceRepository.save(new JobServices(repair, service, quantity));
    }
}
```

The service layer is responsible for enforcing all business rules, so any logic required to guarantee data integrity must be implemented there. Examples include verifying that a repair or service exists before creating a *JobService* instance, and ensuring that the quantity of a part or service on a repair can never drop below zero.

Although the input field in the UI uses a minimum value of zero to prevent negative quantities from being submitted, this client-side validation is not sufficient on its own. The service layer still performs all necessary checks to ensure the system remains consistent and secure.

Another functionality implemented in the service layer is search. This is used in multiple places in the program to search for repairs, parts, and services. Because the functionality is used in all service components, it is implemented first in a base search interface, which the other service components implement. The code snippet below shows this base search method

```
public interface BaseSearchService<T> {
    List<T> search(String keyword);
}
```

Other service components override this method and define how search should be performed for that specific class. Below is the implementation of search in the product service, which uses the *productRepository* to find search matches. The @Override annotation is used in Spring Boot when a subclass is replacing inherited or behavior from superclasses or implemented interfaces, such as in this case.

```
// Custom Search Method for Products - HELPER METHOD
@Override
public List<Product> search(String keyword) {
    if (keyword == null || keyword.isBlank()) {
        return productRepository.findAll();
    }
    return productRepository.search(keyword);
}
```

This search method first checks whether a keyword is provided in the method call. If not, the method returns all products. If the method is called with a keyword, the *productRepository* is used to search for a given keyword using a custom search method shown below. This method from the *productRepository* performs a search in products. It checks whether the search term appears in any of several fields, including the product's ID, product number, name, EAN, or type, using case-insensitive matching. Products that match the keyword in any of these fields are returned, sorted alphabetically by name.

```
@Query(
  """
    SELECT p FROM Product p
    WHERE CAST(p.id AS string) LIKE CONCAT('%', :kw, '%')
       OR LOWER(p.productNumber) LIKE LOWER(CONCAT('%', :kw, '%'))
       OR LOWER(p.name) LIKE LOWER(CONCAT('%', :kw, '%'))
       OR LOWER(p.EAN) LIKE LOWER(CONCAT('%', :kw, '%'))
       OR LOWER(p.type) LIKE LOWER(CONCAT('%', :kw, '%'))
    ORDER BY p.name ASC
    """
)
List<Product> search(@Param("kw") String keyword);
```

Search functionality in the other service classes is implemented using simple single-field lookups. For repairs and services, the search is performed by matching only the title field. This is done using Spring Data JPA derived query methods, such as the following example used when searching for a repair:

```java
List<Job> findByTitleContainingIgnoreCase(String keyword);
```

A more sophisticated search, like the one for products above, could be implemented in the program for a better user experience, also letting the user search for a repair using a customer name, phone or email. Although this was left out to focus on meeting the key requirements of the program first.

Tests, described in the Section 8 section, verify that these methods correctly enforce every business rule.

The next section describes how the repository pattern is implemented in the application, which the service components use to read and write data to the database.

7.2.4 Repositories Using JPA

A JPA Repository is a Spring Data interface that abstracts the data-access layer of the application. It eliminates the need for manual SQL or boilerplate CRUD code by automatically generating queries and managing persistence.

Key characteristics:

- Extends `JpaRepository<T, ID>`
- Provides ready-made CRUD operations (Create, Read, Update, and Delete)
- Supports query creation from method names
- Integrates with JPA/Hibernate to map Java objects to database tables

Example:

```java
public interface JobRepository extends JpaRepository<Job, Integer> {}
```

How JPA Methods Are Created and How They Interact with the Database

Spring Data JPA can generate queries automatically based on method names.

Example:

```java
List<Job> findByTitle(String keyword);
```

Process:

1. The method name is parsed (e.g., `findByTitle`).
2. Spring Data constructs the appropriate JPQL/SQL query.
3. Hibernate executes the query on the database.
4. The results are returned as Java entity objects.

7.2.4.1 Entities & One-to-Many Relationships

Entities

An entity is a Java class mapped to a database table. It uses the `@Entity` annotation and includes a primary key marked with `@Id`.

Example:

```java
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
```

```java
    @Column(name = "\"productNumber\"")
    private String productNumber;

    private String name;

    @JsonProperty("EAN")
    @Column(name = "\"EAN\"")
    private String EAN;

    @Column(name = "category")
    private String type;

    private Double price;
```

One-to-Many Relationship

A One-to-Many relationship exists when one parent entity is associated with multiple child entities. This is used to create join-tables, such as in the example below for the *jobPart* join-table. Here, one part can be associated with many repairs, and one repair can be associated with many parts.

Example:

```java
@Entity
@Table(name = "job_part_jointable")
public class JobPart {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @ManyToOne
    @JoinColumn(name = "job_id", nullable = false)
    private Job job;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;
```

This results in a foreign key `job_id` and `product_id` in the `job_part_jointable` table.

7.2.4.2 Why use JPA?

The Java Persistence API (JPA), specifically Spring Data JPA, is used for this project because it significantly simplifies the data-access layer of the application. Instead of manually writing SQL statements or implementing repetitive CRUD operations, JPA automates these tasks and allows for working with database tables as standard Java objects.

Benefits to JPA

- **Reduced boilerplate:**
  Standard operations (save, delete, find, update) are already implemented in JpaRepository, eliminating the need for custom DAO (Data Access Object) classes.
- **Automatic query generation:**
  Spring Data constructs queries based on method names (e.g., findByTitle), reducing the need for manual SQL.
- **Simplified database connection and management:** JPA handles entity mapping, transactions, and connection lifecycle through Hibernate.

- **Clear and maintainable architecture:**
  The repository pattern keeps business logic and data-access code cleanly separated.

7.2.5 Database & Docker

The system relies on structured data and consistent database behavior, which makes PostgreSQL a suitable relational database choice. Running PostgreSQL inside Docker provides a stable and reproducible development environment, ensuring that all instances of the project use the same configuration and behave consistently across different machines.

7.2.5.1 Choice of Database Technology

Relational Database

A relational database management system (RDBMS) was chosen for this project due to the structured, predictable, and highly interconnected nature of the data in the desired bike-repair system. Core entities such as *Repairs*, *Parts*, and *Services* have clearly defined relationships, including one-to-many and many-to-many associations. These relationships are well-defined and require strict data consistency, which relational databases ensure through ACID transactions (a set of rules/properties, *Atomicity*, *Consistency*, *Isolation*, *Durability*, that guarantee reliable and predictable database operations).

ACID describes the four guarantees a relational database provides during transactions. *Atomicity* ensures each transaction completes fully or not at all. *Consistency* ensures the database always moves between valid states and respects constraints. *Isolation* prevents transactions from interfering with one another when executed simultaneously. *Durability* guarantees that once a transaction is committed, the data is permanently saved, even in the case of crashes [11]. Together, these properties ensure predictable, correct updates, which are essential in a repair system where accuracy is critical.

Furthermore, the repair workflow depends strongly on maintaining referential integrity. For example:

- A repair should never reference parts or services that no longer exist.
- Deleting a repair should also remove associated part/service join-table entities.

Relational databases provide mechanisms, such as foreign keys, constraints, and cascading rules, to ensure this integrity without requiring additional logic to be implemented. This made an RDBMS a natural choice over other non-relational databases.

Why PostgreSQL?

PostgreSQL was chosen because it integrates cleanly with Spring Boot and JPA (Java Persistence API), is open-source, and provides reliable ACID-compliant transaction handling. This is important in a repair system where prices, parts, and statuses must always remain consistent. PostgreSQL also offers referential integrity tools, stable performance, and a simple setup when running in a Docker container. This made it a practical and safe choice for the project.

7.2.5.2 Using Docker for the Database

Docker is used to run the PostgreSQL database in an isolated environment. Instead of installing PostgreSQL manually, the entire setup is defined in a `docker-compose.yml` file. This ensures that the database always runs with the same version, configuration, and credentials, regardless of which machine the project is running on. Starting the database requires only a single command, making the setup easy to reproduce during program development. Running PostgreSQL inside Docker also isolates the database from the host system, preventing port conflicts and avoiding local installation issues.

The PostgreSQL container is based on the `postgres:16` image[3] and is configured using environment variables from the `.env` file. This prevents credentials from being hardcoded in the compose file or the source code. A Docker volume named `db_data` stores the actual database files so that data is preserved even if the container is restarted. The `./db/init` folder is attached as an initialization directory, allowing SQL scripts to run automatically the first time the database starts. The containers communicate using Docker's default bridge network, allowing services to reference each other by name.

A health check is set up using `pg_isready`. This ensures that the database is fully ready before other services begin interacting with it. To support development, an additional Adminer container is included. Adminer provides a simple web interface for viewing tables and running SQL queries, available on port `8080`. An example of this web interface can be seen in Figure 23.

`docker-compose.yml` File:

```yaml
services:
    db:
        image: postgres:16
        container_name: localdb-postgres
        restart: unless-stopped
        ports:
            - "${POSTGRES_PORT:-5432}:5432"
        environment:
            POSTGRES_USER: ${POSTGRES_USER:-app}
            POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-app}
            POSTGRES_DB: ${POSTGRES_DB:-appdb}
        volumes:
            - db_data:/var/lib/postgresql/data
            - ./db/init:/docker-entrypoint-initdb.d:ro
        healthcheck:
            test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
            interval: 5s
            timeout: 5s
            retries: 10

    adminer:
        image: adminer:4
        container_name: localdb-adminer
        restart: unless-stopped
        depends_on:
            - db
        ports:
            - "${ADMINER_PORT:-8080}:8080"

volumes:
    db_data:
```

[3]Additional information about the Docker Architecture can be found in Section 6.2.6.

Figure 23: Adminer's Web Interface for Database Management Showing *Servces* Data Table

The application connects to the PostgreSQL container using the JDBC URL (Java Database Connectivity URL) from the `.env` file. When the application starts, Hibernate generates the database schema based on the entity classes, keeping the structure aligned with the domain model.

`.env` File:

```
JDBC_DATABASE_URL=jdbc:postgresql://localhost:5432/admin
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin
```

Hibernate is the JPA implementation used in the application, responsible for mapping Java entity classes to relational database tables and handling the communication between the application and PostgreSQL. Hibernate is configured with `spring.jpa.hibernate.ddl-auto=update`, which instructs the framework to automatically create or adjust database tables to match the entity classes during development. This keeps the database synchronized with the domain model without requiring manual SQL transfers. This setting is intended for development use, as it allows modifications such as new fields or relationships to be reflected in the database each time the application starts. This configuration is defined in the `application.properties` file, which contains the application's runtime settings, including database connectivity and Hibernate behavior.

`application.properties` File (Snippet):

```
...
# --- Data Base Configuration --- #
spring.datasource.url=${JDBC_DATABASE_URL}
spring.datasource.username=${POSTGRES_USER}
spring.datasource.password=${POSTGRES_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
# JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.jdbc.lob.non_contextual_creation=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
...
```

7.3 Front-End

The second part of the implementation section describes the implementation of the system's front end. It covers the technologies and components used to present data to the user, handle client-side interactions, and ensure a responsive and intuitive user experience. This includes the use of *Thymeleaf* for generating dynamic HTML templates, the application of *Bootstrap* for consistent and adaptable styling, and the implementation of client-side functionality for communicating with the back-end.

Furthermore, this part also details the integration of the *FullCalendar* library, which is used to visualise scheduled repairs by dynamically retrieving and displaying job data as calendar events.

7.3.1 HTML Templates Using Thymeleaf

Thymeleaf is a Java template engine for XML, XHTML, and HTML5 [6]. Thymeleaf is best suited for use in MVC applications, and it is commonly used in Spring Boot applications.

Thymeleaf makes it easier to generate dynamic HTML pages by allowing developers to embed back-end data directly into the HTML using simple attributes, whilst keeping the templates valid HTML files that can be viewed and edited normally. Together with Spring Boot, Thymeleaf renders the page on the server and serves it directly to the user as an HTML page. This results in faster load times and better security.

Below is an example of how Thymeleaf is used in the application. In the snippet, Thymeleaf attributes such as *th:text* are used to inject dynamic values from the back-end directly into the HTML table. Each table row displays a specific field from the job object, such as its status, creation date, duration, and customer information. For instance, the job's status is inserted using *th:text="{job.status.name}"*, and the creation date is formatted with the built-in #temporals.format(...) utility.

Because Thymeleaf evaluates these expressions on the server, the resulting HTML sent to the client contains fully rendered values with no additional client-side processing required. This approach keeps the markup clean and readable, while still allowing the UI to reflect dynamic data.

```html
<!-- Table content: shows job fields populated by Thymeleaf -->
<div class="table-content-container">
    <table class="table tableRemoveLastLine mb-1">
        <tbody>
            <tr>
                <td><b>Status:</b></td>
                <td>
                    <span
                        class="job-status"
                        id="job-status"
                        th:text="${job.status.name}"
                    >
                    </span>
```

```
                </td>
            </tr>
            <tr>
                <td><b>Oprettet:</b></td>
                <td
                    th:text="${#temporals.format(job.date, 'd. MMMM yyyy HH:mm')}"
                >
                </td>
            </tr>
            <tr>
                <td><b>Varighed:</b></td>
                <td th:text="${job.duration} + ' minutter'"></td>
            </tr>
            (...)
        </tbody>
    </table>
</div>
```

7.3.2 Bootstrap Styling

Bootstrap is a front-end framework that utilizes CSS, HTML, and JavaScript-based design templates for simplifying the process of creating a user interface. It is beneficial when creating user-friendly interfaces due to its predefined layout components, ensuring a uniform appearance [12].

Bootstrap includes a large file of approximately 10,000 lines of code, featuring a wide range of CSS and JavaScript, which can lead to potentially bloated code, resulting in slower load times for a website. Bootstrap can contain a large amount of unnecessary code, especially for smaller systems, that might not need all the predefined components in Bootstrap, but since writing CSS in itself is a largely time-consuming process, when developing a system under a time constraint and with an emphasis on core functionality, Bootstrap is a strong choice. Moreover, Bootstrap also allows for easy modifications of its predefined layout components, enabling developers to accommodate customers' user interface requirements [12].

The code snippet below shows an example of how Bootstrap is used in the application. The code is from the *jobListe* HTML file, specifically the table where jobs are rendered as a list. The outer <table> tag uses Bootstrap's *.table* and *.table-hover* classes, which apply a set of predefined CSS styles to the table element. Bootstrap also provides predefined styling for the table header and table body. This example also used thymeleaf for dynamic rendering like described in Section 7.3.1

```
<table class="table table-hover align-middle mt-lg-3">
      <thead>
          <tr>
          (...)
          </tr>
      </thead>
      <tbody id="table-body">
          <tr
              th:each="job : ${jobs}"
              th:attr="data-href=@{/jobliste/{id}(id=${job.id})}"
              style="cursor: pointer"
          >
              <td class="w-15">
```

```
                <div class="d-flex flex-column gap-1">
                (...)
                </div>
            </td>
            (...)
        </tr>
    </tbody>
</table>
```

7.3.3 Client-Side Functionality

The system implements dynamic client-side functionality using JavaScript, which interacts with the Spring Boot back-end via HTTP requests. When a user performs actions such as adding, editing, or deleting jobs and products, JavaScript event listeners trigger functions that send asynchronous requests using the fetch API to REST endpoints defined in the server-side controllers. These requests utilize standard HTTP methods to facilitate communication with the server.

The back-end processes these requests, updates the database as required, and returns responses such as JSON data or status codes. Client-side scripts then update the user interface according to the server's response, enabling real-time feedback and a seamless user experience without requiring full page reloads. This architecture supports efficient communication between the front-end and back-end, and enables features such as modal dialogs, dynamic tables, and error handling directly in the browser.

In the example below, a button is provided in HTML for deleting a job. The button includes a data attribute with the job's ID.

```
<button class="btn btn-danger" id="deleteBtn" th:attr="data-job-id=${job.id}">Slet
Reparation</button>
```

The corresponding JavaScript listens for a click event on this button, retrieves the job ID, and sends an HTTP DELETE request to the back-end.

```
document.getElementById('deleteBtn').addEventListener('click', function() {
    const jobId = this.getAttribute('data-job-id');
    if (confirm('Are you sure you want to delete this job?')) {
        fetch('/api/jobs/' + jobId, { method: 'DELETE' })
            .then(resp => {
                if (resp.ok) {
                    window.location.href = '/jobliste';
                } else {
                    alert('Could not delete job');
                }
            })
            .catch(() => alert('Could not connect to server'));
    }
});
```

Then the Spring Boot controller handles the DELETE request.

```
@DeleteMapping("/api/jobs/{id}")
@ResponseBody
@Transactional
public ResponseEntity<Void> deleteJob(@PathVariable Integer id) {
    if (!jobRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
```

```
    jobPartRepository.deleteByJobId(id);
    jobRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}
```

This example demonstrates how client-side JavaScript is used to send an HTTP DELETE request to the back-end when a user deletes a job. The front-end retrieves the job ID from HTML, communicates with the REST API using the fetch function, and updates the user interface based on the server's response. The back-end controller processes the request, modifies the database, and returns an appropriate status code, enabling smooth interaction between client and server.

7.3.4 Calendar implementation using FullCalendar

The FullCalendar library is utilized to provide an interactive and visually appealing calendar interface within the website. FullCalendar is a JavaScript library that enables the display, management, and manipulation of calendar events directly in the browser. The library is especially useful in a project with a deadline, where building a calendar from the bottom, is not necessary.

In this implementation, FullCalendar is integrated into the relevant HTML template and initialized via client-side JavaScript. The calendar displays scheduled jobs or other time-based data received from the back-end through HTTP requests. Users can view events in various formats, such as day, week, and month, and interact with the calendar to inspect details or trigger further actions. This approach enhances usability by allowing users to manage and visualize time-based information efficiently within the application.

```
<div id="calendar"></div>
<script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></
script>
<script src="/js/calendar.js"></script>
```

The code above demonstrates how the FullCalendar is rendered in the HTML element with the ID *calendar*. Below is a demonstration of how it is initialized in JavaScript. Configuration options specify the initial view, localization, event source, and toolbar layout. The *eventClick* callback enables the user interaction with calendar events, such as displaying details or triggering further actions.

```
document.addEventListener('DOMContentLoaded', function() {
    var calendarEl = document.getElementById('calendar');
    var calendar = new FullCalendar.Calendar(calendarEl, {
        initialView: 'dayGridMonth',
        locale: 'da',
        events: '/api/calendar/events',
        headerToolbar: {
            left: 'prev,next today',
            center: 'title',
            right: 'dayGridMonth,timeGridWeek,timeGridDay'
        },
        eventClick: function(info) {
            alert('Event: ' + info.event.title);
        }
    });
    calendar.render();
});
```

7.3.5 Example Use-Case of Adding a Product to a Repair

To illustrate how the system's architectural layers interact during runtime, this section presents an end-to-end example of the data flow that occurs when a user creates a new service through the "Add Product" modal on the *Product Page*.

This example demonstrates how data flows from the front-end, through the controller, and optionally also the service layer, into the repository layer, and finally into the PostgreSQL database.

Front-End - User Input & JavaScript Fetch Request

When the user fills out the service form in the modal and presses "*Tilføj Ydelse*", the JavaScript file `addProductsModal.js` gathers the values and sends them to the back-end as JSON using the Fetch API:

```
// Gather input values from the service form
const serviceData = {
    name: document.getElementById('service-navn-text').value,
    price: document.getElementById('service-price').value,
    duration: document.getElementById('service-duration').value,
};
try {
    const response = await fetch('/api/services', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(serviceData),
    });

    // Check for fetch errors and handle them gracefully
    await handleFetchErrors(response);

    // Reload the page to show the newly added service
    window.location.reload();
}
catch (error) {
    // Log any network/server errors
    console.error('Error creating service:', error);
}
```

The `POST` request is sent to the endpoint `/api/services`, where it is handled by a Spring REST Controller.

Controller Layer - Mapping JSON Into a Java Entity

The `POST` request is received by the REST controller responsible for handling service-related API calls. Spring automatically converts the incoming JSON payload into a `Services` entity through the `@RequestBody` annotation. Instead of interacting with the database directly, the controller delegates the operation to the service layer, ensuring clear separation of concerns:

```
// Create Service - CREATE
    /** @param service the service object sent from frontend to create **/
    /** @return ResponseEntity containing the created service **/
    @PostMapping
    public ResponseEntity<Services> createService(@RequestBody Services service) {
        // Call the createService method from the service layer
        Services saved = serviceService.createService(service);
```

```
        // Return HTTP 200 OK with the created service object as JSON
        return ResponseEntity.ok(saved);
    }
```

The controller handles API communication only and leaves domain logic and database interaction to the service layer, improving maintainability and structure.

Service Layer - Business Logic & Validation

The service layer acts as the middleman between controllers and repositories. At the current stage of the project, creating a service does not involve complex logic, so the method primarily forwards the entity to the repository for storage. This structure, however, allows easy extension in the future, such as validating duration, preventing duplicates, or formatting service names before saving.

```
// Create Service - Create
    /** @param service the Service object to save **/
    /** @return the saved Service **/
    public Services createService(Services service) {
        // Save the new service using the repository
        return serviceRepository.save(service);
    }
```

By placing this logic in the service layer rather than the controller, the system follows the layered architecture principles, ensuring controllers remain compact and focused entirely on request handling.

Repository Layer - Persisting the Entity via JPA

Persistence is handled by the `ServiceRepository`, which extends Spring Data JPA's `JpaRepository`. Because of this, CRUD functionality is generated automatically:

```
public interface ServiceRepository extends JpaRepository<Services, Integer> {
    // Search by Name
    List<Services> findByNameContainingIgnoreCase(String name);

    // Spring Data auto-provides findAll(), findById(), save(), delete(), etc.
}
```

When `save()` is called:

1. Hibernate inspects the `Services` entity.
2. It generates an SQL `INSERT` statement.
3. It sends that SQL statement to the PostgreSQL database.
4. It returns the saved entity, including the auto-generated primary key.

Database Layer - Hibernate Generates SQL

The `Services` entity class defines how the object maps to the database table:

```
// Represents a service in the system.
// This entity is mapped to the "services" table in the database.
@Entity // Marks the class as a JPA entity (Maps to a database table)
@Table(name = "services") // Explicitly sets the table name in the database
public class Services {
    // Attributes
    @Id // Marks this field as the table's primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-incremented ID
    private int id; // Maps to: id (Generated by default as Identity)

    private String name;  // Name of the service
```

```java
    private double price; // Price of the service
    private int duration; // Duration of the service in minutes

    ...
}
```

When the repository calls `save()`. Hibernate generates SQL similar to:

```sql
INSERT INTO services (name, price, duration)
VALUES (`Gear Justering`, 199.0, 20);
```

The PostgreSQL database stores the record and returns the newly generated `id`, completing the request cycle.

This data flow ensures a clear separation of concerns, where controllers handle requests, the service layer contains business logic, and the repository layer manages interactions with the database.

8 Testing

This section describes the testing activities carried out to ensure the correctness, stability, and usability of the system, and ultimately to verify that the application fulfills the must-have and should-have requirements specified in Section 2.4. The testing approach combines automated tests with user testing to validate both technical functionality and real-world applicability.

Automated testing is performed using *JUnit*, focusing first on unit tests for the JPA entities to verify that the domain model behaves as expected. In addition, we include targeted tests for the *JobController*, ensuring that its API endpoints respond correctly and handle requests according to the application's requirements. To validate the interaction between components, integration tests are conducted, testing the system end-to-end from the controller layer through to the database.

Finally, a user evaluation is carried out to gather feedback on the application's usability, design, and functionality. This provides insight into how well the system meets the needs of its intended users and highlights areas for potential improvement. Together, these testing activities help ensure that the system is both technically sound and aligned with user expectations.

8.1 Unit Testing

This section outlines the JUnit-based unit tests used to validate the behavior of the JPA entities and the *JobController* in isolation. JUnit is one of the established and widely used testing frameworks for Java code, which is why it has been chosen as the preferred testing framework.

The first part of this section describes unit testing of the JPA entities used in the application.

8.1.1 JPA Entities

To test the JPA entities and the service logic that depends on them, the system uses *JUnit 5* together with *Mockito*. Since unit tests should focus on the behavior of a single component in isolation, the JPA repositories are mocked rather than connected to a real database. This ensures that the tests remain independent of external systems.

The repository interfaces (JobRepository and JobPartRepository) are annotated with @Mock, allowing their responses to be simulated. The JobService under test is annotated with @InjectMocks, which automatically injects the mocked repositories into the service, as seen in the code snippet below.

```
@Mock
private JobRepository jobRepository;

@Mock
private JobPartRepository jobPartRepository;

@InjectMocks
private JobService jobService;
```

```
private Job testJob;
private Product testProduct;
```

The @BeforeEach annotation is used to automatically define a test repair and a test product to be used in testing. A new test repair, and test product is created before each test, to ensure fresh objects without any manipulations.

```
@BeforeEach
    void setUp() {
        testJob = new Job();
        testJob.setId(1);
        testJob.setTitle("Test Job");

        testProduct = new Product("P001", "Test Product", "1234567890123",
        "Electronics", 99.99);
    }
```

When defining tests, such as the one in the code snippet below expectations are defined with statements such as *when(jobRepository.findById(1))*, the tests control the returned data and can verify how the service handles different scenarios, for example when a job is found, when no job exists, or when an empty or null search query is provided. Assertions are then used to confirm that the service responds correctly, and *verify()* checks ensure that the service interacts with the repositories as expected.

```
@Test
    void testGetJobById_Success() {
        when(jobRepository.findById(1)).thenReturn(Optional.of(testJob));

        Job result = jobService.getJobById(1);

        assertNotNull(result);
        assertEquals(1, result.getId());
        assertEquals("Test Job", result.getTitle());
        verify(jobRepository, times(1)).findById(1);
    }
```

In total, tests have been defined for five of the six entities in use in the program, as the *Product* and *Services* classes are very simular. For each entity creating, setters, getters are tested, as well as methods for entities that have methods defined.

The next part of testing describes testing of more complex methods in the JobController.

8.1.2 JobController

This section presents the unit tests written for the JobController, verifying that its REST endpoints behave correctly under different conditions.

The JobControllerTests verify all seven REST endpoints of the JobController by treating it as a thin HTTP layer, the tests use Spring's MockMvc to simulate real HTTP requests and responses, while the underlying JobService is fully mocked with Mockito so that no actual business logic or database access is executed. Each test ensures that incoming requests are correctly handled, mapped, and forwarded to the service with the expected arguments. Both successful (happy-path) and failure responses such as 200 OK, 204 No Content, 400 Bad Request, and 404 Not Found are validated to confirm that the controller returns the appropriate status codes and JSON output. The tests cover job retrieval, creation, full updates, partial description updates, product assignment and removal, and deletion, ensuring that the controller behaves reliably across all operations while remaining a thin delegation layer.

Below is a code snippet from one of the unit tests in JobControllerTests.

```
@Test
void shouldReturn204WhenJobDeleted() throws Exception {
    // Mock service to simulate successful deletion
    when(jobService.deleteJob(42))
            .thenReturn(ResponseEntity.noContent().build());

    // Perform DELETE request and verify response
    mockMvc.perform(delete("/api/jobs/{id}", 42))
            .andExpect(status().isNoContent());

    // Confirm controller delegated to the service layer
    verify(jobService).deleteJob(42);
}
```

This test verifies the controller's behavior when deleting a job. The JobService is mocked to return a 204 No Content response, meaning the job was successfully removed. The test issues a DELETE request to /api/jobs/42 using MockMvc and checks that the controller returns the same 204 status to the client. Finally, verify(jobService).deleteJob(42) ensures that the controller correctly forwarded the job ID to the service, in this case, "42" just being a sample/placeholder job id.

8.2 Integration Testing

This section describes the integration tests used to verify that the system's components work together correctly as a whole. Unlike unit tests, integration tests run on real application layers, including the controller, service, and database, ensuring that API endpoints and business logic function as expected in an integrated environment. These tests provide confidence that the system behaves reliably under realistic conditions.

Integration tests are defined to test four of the main use-cases of the application from Section 4.1. These are:

- Creating a repair
- View repair diagnostic
- Add a service to a repair
- Add a part to a repair

The use-case *"Create invoice for POS system"* is not included in this test, as it is not a function in the current state of the application. How well the application fulfills the requirements is discussed further in the discussion chapter.

Test Setup JUnit is used as the testing framework together with Spring Boot's test dependencies, like in unit testing. This allows to use Spring Boot annotations while tests are being powered by JUnit. To not affect the main database, a test database container was created in Docker to run integration tests. Before running each test, the database schema is created from scratch, user hypernate, and when the test is finished, the schema is dropped so the database is empty and ready for another test. This ensures that tests are independent of one another.

The section below describes how integration testing was done for one of the use-cases, creating a new repair.

Creating a repair

When creating a repair, the user sends an HTTP request from the user interface with repair information to the create repair API endpoint defined in the *JobController*. From here, the back-end handles the information and adds a new repair to the database, and also creates an instance in the *JobService* join-table for any services added to the repair during creation.

The test defined below tests this integration. First, the test method defines a payload to send to the create job API endpoint. This payload is a job object with a status and a service attached.

```
Map<String, Object> payload = new HashMap<>();
        // Job object:
        payload.put("title", "Test repair 1");
        payload.put("customer_name", "John Doe");
        payload.put("customer_phone", "12345678");
        payload.put("job_description", "Broken chain");
        payload.put("work_time_minutes", 30);
        payload.put("price_per_min", 5.0);
        payload.put("duration", 60);
        payload.put("date", "2025-10-28T13:45:00");

        // Create a Status object
        Map<String, Object> statusObj = new HashMap<>();
        statusObj.put("id", 1);
        payload.put("status", statusObj);

        // Create a Service to add
        Services testpart = new Services("fuld service", 200.00, 60 );

        ResponseEntity<Map> productResp =
                restTemplate.postForEntity(
                        "http://localhost:" + port + "/api/services",
                        testpart,
                        Map.class
                );

        assertEquals(HttpStatus.OK, productResp.getStatusCode());
        serviceId = (Integer) productResp.getBody().get("id");
        assertNotNull(serviceId);

        // Add the Service to the Repair
        repair.put(
                "services",
                List.of(
                        Map.of(
                                "id", serviceId,
                                "quantity", 2
                        )
                )
        );
```

In the next part, an *HttpHeaders* object is created, and the *Content-Type* header is explicitly set to *application/json*. This tells Spring that the request body contains JSON data. Then headers and the payload are wrapped inside an *HttpEntity*. When the request is sent, Spring automatically serialises the payload map into a JSON object and includes it in the body of the HTTP request. Finally, the request is sent to the */api/jobs/create* endpoint using the generated URL.

```java
HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_JSON);
HttpEntity<Map<String, Object>> entity = new HttpEntity<>(payload, headers);
String url = "http://localhost:" + port + "/api/jobs/create";

// Call the endpoint
ResponseEntity<Map> response = restTemplate.postForEntity(url, entity, Map.class);
```

Now the repair and jobService instance should be created in the database. This is tested using the *jobRepository* and *jobServiceRepository* to access the database tables of the test database, as seen below.

```java
// Validate Job in DB
Job saved = jobRepository.findById(jobId).orElse(null);
assertNotNull(saved, "Job was not inserted into database");

assertEquals("Test repair 1", saved.getTitle());
assertEquals("John Doe", saved.getCustomer_name());
assertEquals(30, saved.getWork_time_minutes());
assertEquals(5.0, saved.getPrice_per_minute());


// Validate Service join-table
List<JobServices> jobServices = jobServiceRepository.findByJobId(jobId);
assertFalse(jobServices.isEmpty(), "No JobServices rows created");

JobServices js = jobServices.getFirst();

assertEquals(1, js.getService().getId());
assertEquals("Test repair 1", js.getJob().getTitle());
assertEquals("fuld service", js.getService().getName());
assertEquals(2, js.getQuantity());
```

Integration Test Evaluation

The other integration tests were created using the same method as the example above and ran against the same test database. All integration tests passed without errors. However, during the process of writing them, several design issues became visible. The API endpoints described in Section 7.2.2 function correctly according to the tests, but they require data to be passed in different formats. The *createJob()* endpoint expects a Map, adding or removing products from a repair requires a List, and updating a job requires a complete repair entity as input. Since the API endpoints were designed individually based on immediate needs, no consistent guideline or standard was followed regarding how data should be formatted.

This inconsistency can create confusion for developers and testers when maintaining or extending the system. To improve this part of the program and establish a clear standard across all endpoints, DTOs (Data Transfer Objects) could be introduced. DTOs would simplify the code, enforce consistent input structures, and also improve security by controlling which data can be received and sent through the API.

After ensuring the application worked as intended, the users at Performsport were included to verify that the application provided real value. This was done by conducting a final user evaluation, described in the following section.

8.3 User Test

Usability testing is an important method for assessing how effectively and intuitively users can interact with a system. It focuses on observing real user behavior as they attempt to complete representative tasks, helping identify usability issues and revealing opportunities for improvement. Conducting such tests ensures that the interface aligns with user expectations and supports the iterative refinement of both functionality and design.

In this project, usability testing was carried out once a functional prototype had been developed and core features were in place. Performsport had already provided feedback on earlier design iterations, which helped shape both the low-fidelity and high-fidelity prototypes. To evaluate the implemented system, a final usability assessment was conducted at the end of the implementation phase.

For this evaluation, Performsport was asked to complete a set of representative tasks (See testing plan in appendix in Section 13.3), including: scheduling a repair, defining a new service, and adding that service to an existing repair. They completed all tasks without difficulty and found the application intuitive and easy to use. Aside from expressing a desire for a search function allowing repairs to be found by customer phone number or name, they were highly satisfied with the solution. An extended search functionality has now been implemented in the program based on this user evaluation. This feature should have been prioritized from the beginning, as it was one of the requirements that Performsport mentioned during the initial interview.

One issue identified was related to the calendar interface: short-duration repairs display a cropped title due to limited space. This could be resolved by reducing the font size for short events or displaying the information in an alternative format.

Although it is difficult to determine the application's full external value based on this small test alone, the insights gained from this usability evaluation highlight both the strengths of the current solution and the areas where improvements can be made. Further usability testing should be conducted to validate the application and ensure that it is fully fit for use. These tests could include performing additional usability evaluations similar to those described in this section, or ideally, allowing Performsport to integrate the system into their daily workflow to assess how well it functions in a real-world context.

8.4 Assessment of Requirements After Testing

The testing described in this chapter verifies that the application fulfills all of the *Must have* and *Should have* requirements from the MoSCoW specification defined in Section 2.4. The application does not support any of the features labeled as *Could have*, as the development effort focused on implementing the core functionalities that deliver the most value to the user within the project's scope.

Some requirements were verified through automated unit and integration tests using JUnit, while others were validated directly by users during the user testing described above. Although the test suite could be expanded to cover additional edge cases within the problem domain, and more time could be spent researching and identifying such cases, the current tests sufficiently validate the application for its intended use.

The system functions in Section 4.3 are ranked by complexity, and the section concludes that functions with higher complexity should be prioritized in testing. In this model, *create a new repair* and *change calendar view mode* are the most complex functions. The creation of a repair is covered in several tests and is therefore well represented.

However, changing the calendar view mode is not included in the automated tests, as this feature is handled internally by the *FullCalendar* framework. It is assumed that this functionality is already tested as part of the framework, which is why it was given lower testing priority. The feature is still verified in the user tests, where users confirmed that the calendar displays repairs correctly and that switching between views works as expected.

These findings form a basis for the discussion in the following section.

9 Discussion

Developing a web-based bicycle repair management application revealed valuable insights and challenges that extend beyond the implementation. This chapter reflects on the practical and technical aspects, considering the solution's requirements, design choices, and limitations. This chapter also highlights areas for improvement, possible future extensions, and lessons learned throughout development, including testing and technical challenges. Finally, both the internal quality of the system and the external value it provides to Performsport are evaluated.

9.1 Project Requirements

As previously mentioned in the testing chapter, the program fulfills all *Must Have* and *Should Have* requirements defined by the MoSCoW method. All core functionalities required for the application to operate correctly have been implemented and tested successfully. However, none of the *Could Have* requirements have been implemented, as they were considered to be of lower priority and outside the scope of the project.

The application was developed with a clear focus on the design criteria ranked as *very important*, ensuring that the most critical aspects were addressed first. Usability and comprehensibility guided the majority of design decisions, with the primary goal being to create a solution that aligns naturally with Performsport's existing workflow.

The *important* criteria were also strongly supported throughout the implementation. The system performs efficiently in a local environment, follows the functional requirements, and maintains consistent behavior during core operations.

The *less important* criteria were only fulfilled to the extent necessary for the project's scope, and thus were not given primary attention during implementation.

Overall, the development focused on the most crucial criteria, while still meeting the important requirements and reasonably addressing the lower-priority considerations.

9.2 Improvements to Architectural & Oriented Design

As described in Section 7.1 the architectural design of the application changed during the implementation phase of the project. Changes were made both to the overall system architecture and to the class structure. This section discusses some of the choices made during this process.

Product Super-class for Parts & Services

The decision to remove the *Product* superclass made sense in this context to simplify the class hierarchy. However, implementing a superclass for *Part* and *Service* could also offer benefits. It would allow for polymorphism and potentially simplify the service layer by enabling the application to use a single method for creating or removing a product, and a single method for adding or removing products to a repair, instead of having separate methods for both *Part* and *Service* objects that perform similar tasks.
Without a shared superclass, the service layer must differentiate between *Part* and *Service*,

resulting in duplicated methods and decreased flexibility. This will also make the code harder to maintain in the future.

Despite these advantages, the decision not to implement the superclass was based on the relatively low complexity of the domain model, with only two classes, and the wish not to use abstraction when it was not necessary.

Implementing Design Patterns
The application uses several common design patterns, which are described throughout the report. These include the layered architecture pattern and the repository pattern. The Spring framework also applies several design patterns internally. One of the most important is dependency injection (DI), which is used to automatically provide services and repositories to the classes that depend on them. Instead of creating these objects manually [13]. Spring also uses the singleton design pattern for bean creation, to make sure only one instance of each service and controller components exists in the application [14].

Other design patterns were considered during implementation, but due to the simplicity of the problem domain and the existing features of Spring Boot, implementing these design patterns would add minimal value to the application. Design patterns considered included:

- Using the **Strategy** pattern for search, by defining strategies to search for title, price, customer name, etc., based on each specific case. But the Spring repository methods made this job very simple already.
- Using the **Factory** pattern for part and service creation. This would involve implementing the *Product* superclass for parts and services, however, since this class was removed during implementation, due to the domain being too simple, this was not possible.
- Using the **Builder** pattern for *Job* creation. This would make creating jobs simpler if not all data fields are required. However, since the application requires all data fields to be filled by the user, and object creation is simple, the builder pattern would add complexity with little upside.

If the application becomes more complex and new features are added, implementing some of these design patterns might be useful to improve maintainability and reduce code duplication.

Mismatched API Endpoints
During testing, another potential improvement to the design became clear. The API endpoints in the controller components all required different input formats, which made them difficult to work with and maintain. As mentioned in Section 8.2, one solution would be to introduce DTOs (Data Transfer Objects) to standardize and simplify the data being passed into the endpoints.

A DTO is a simple object used to transfer data between layers in an application. DTOs are commonly used in API design to ensure a clear and consistent contract between the client and the server. By using DTOs, the application could validate incoming data more reliably and improve security by exposing only the required fields rather than entire domain objects.

Despite these advantages, DTOs were not included in the current implementation. Although they would have improved internal structure and code maintainability, the need for DTOs was identified late in the project, at a point where incorporating them would have exceeded the available time and project scope. Since DTOs primarily enhance the solution's internal value, they were prioritized lower than features that would increase external value and benefit the end user.

Apart from changes to the architectural and object-oriented design, the application could also be improved by adding new features, such as those mentioned in the should have requirements. This is described in the following section.

9.3 Future Developments

There are several areas where the system can be expanded to support Performsport's workflow better and align with the requirements defined in the System Description in Section 2.4. These are discussed here.

Discontinued Status for Parts/Services

One of the most significant limitations is that parts and services cannot be discontinued in the current implementation. As described in the analysis, Performsport prefers discontinuation over deletion, as they want the ability to view past repairs that may reference products that are no longer sold. Implementing a "discontinued" state would therefore improve data integrity and ensure that past repairs remain accessible while preventing discontinued items from appearing in new repair jobs.

GDPR Compliance

Another important aspect concerns GDPR (*General Data Protection Regulation (EU)*) compliance. The system currently stores customer information such as name, phone number, and email, which qualifies as personal data under the GDPR. According to *Article 5* of the regulation, personal data must be processed lawfully, stored securely, and only kept for as long as necessary. Furthermore, *Article 6* requires that personal data is only stored when there is a clear legal basis for it, such as fulfilling a repair order or having explicit customer consent. In addition to defining a lawful purpose, *Article 32* also requires that the stored data is protected through appropriate security measures, for example, access control, encryption, or other safeguards that prevent unauthorized access, such as a log-in system. This means the current implementation does not yet fully meet these requirements. Thus, future development should therefore include GDPR-oriented functionality to ensure that customer data is handled and protected as required by the GDPR [15].

Shopify Integration

A major area of expansion is the integration with Shopify. While the current system operates independently of the POS system, the intended vision is for repairs to be connected to Shopify, allowing draft orders and invoices to be created automatically. At this stage, the system does not support the use case of "*Create invoice from repair for POS system*". This was also noted in Section 8.2, as it fell outside the scope of the implemented solution. Adding Shopify integration would optimize the sales workflow by eliminating manual data entry and ensuring that the repair process merges seamlessly with Performsport's existing POS infrastructure.

User Implications

The absence of these functionalities has several implications for the customer. Without the ability to discontinue products, the product list may become cluttered and disorganized over time, reducing efficiency and increasing the risk of selecting outdated parts and services. The lack of GDPR-specific features means that customer data security relies primarily on server- and database-level protections rather than application-level safeguards, which may not fully satisfy the expectations outlined in the GDPR. Finally, the missing Shopify integration and invoice creation functionality mean that staff must still manually transfer information from the repair system into the POS system, which increases workload and introduces the potential for human error. Implementing these features in future iterations would therefore not only improve operational efficiency but also increase trust, security, and alignment with Performsport's existing digital system.

Together, these possible enhancements show how the system can continue improving to better support both technical requirements and user needs. As these future improvements depend on a stable and well-evaluated foundation, the next section turns to the project's testing strategy and the role it played in shaping the final solution.

9.4 Testing Strategy

Conducting user testing and evaluation early in the development process is beneficial because it allows problems to be identified at an appropriate stage. This prevents the team from having to make large updates or corrections later in the project, which can lead to time-consuming tasks and significant code changes near the end. Early testing can involve users interacting with wireframes, navigation flows, and the overall workflow. Their feedback helps ensure that the system meets user expectations.

Testing each component individually (unit testing) and verifying that these components work together correctly (integration testing) further contributes to system reliability. Since most of our testing was performed near the end of development, many earlier issues had already been resolved manually during implementation. As a result, the final testing phase was relatively smooth and presented few challenges. Early user testing would also have helped verify that the implemented features aligned with actual user needs rather than assumptions.

In conclusion, implementing testing from the beginning of the process leads to a more reliable and user-friendly product and reduces the risk of last-minute issues. A more systematic approach, testing early and testing often, would provide greater confidence in both the stability and usability of the final system. The next section reflects on these challenges and the lessons they brought.

9.5 Coding Challenges

Throughout the system's development, the group faced several challenges related to coding and technical implementation. One of the most significant challenges was the learning curve associated with new frameworks and tools such as Spring Boot, Thymeleaf, PostgreSQL, and a multi-layered back-end architecture. Since the team had little prior experience with these technologies, a considerable amount of time was spent understanding configuration and framework structure before productive development could begin. Although this slowed initial progress, it also provided valuable hands-on experience with industry-relevant technologies.

Working with a multi-layered architecture also presented challenges early in the project. Determining the correct placement of logic was not always straightforward, and several components required restructuring to ensure proper separation of concerns. Over time, the team became more confident with the structure, but the experience emphasized the importance of early architectural planning and maintaining consistency throughout development.

As mentioned in the Section 11, the group was introduced to design patterns and advanced OOP principles late in the semester. Patterns such as Strategy, Factory, and DTO-based structures could have improved maintainability and reduced duplication, especially in relation to parts and services. However, by the time these concepts were introduced, most features had already been implemented, making it impractical to completely refactor the codebase within the remaining time-scope of the project. This highlighted the value of learning architectural principles earlier and inspired the team to apply such patterns more actively in future projects.

Regarding testing, the team also learned that testing should be integrated earlier and more frequently throughout the development process. As noted in the previous Testing Strategy section, Section 9.4, many tests were written near the end of development, meaning bugs were often discovered and fixed manually during implementation instead of being caught automatically. Adopting a "test early, test often" workflow could have optimized debugging and improved reliability.

In summary, the main coding challenges came from learning new frameworks, adapting to a multi-layered architecture, and being introduced to design patterns late in the process. Addressing these challenges contributed to a stronger technical foundation and will inform better design decisions in future software projects.

9.6 Evaluation

The discussion highlights how the developed system meets the essential requirements for supporting Performsport's repair workflow while also revealing areas where the implementation could be improved or extended. It reflects on the architectural decisions made throughout the project, noting that the chosen layered structure and simplified domain model were appropriate for the project scope, even though more advanced abstractions, such as a shared product superclass or DTOs, could have increased maintainability and consistency.

The chapter also identifies several functional limitations that influence the system's external value, including the absence of a product discontinued status, GDPR-related safety measures, and integration with Shopify. These features were considered outside the project's time constraints but remain important for a fully operational solution in Performsport's environment.

In addition, the discussion examines the development process itself. Testing was performed too late in the project, which reduced its effectiveness. Additionally, the introduction of design patterns and architectural principles late in the semester limited the extent to which they could be implemented. The group also encountered a learning curve with new frameworks and technologies, which shaped both the time it took to develop the application and the final architecture.

Overall, the points discussed here form a basis for the conclusion of the project in the next chapter.

10 Conclusion

This project set out to design and implement a digital repair management system that would support Performsport's daily workflow more effectively than their existing combination of handwritten notes and Shopify draft orders. Through analysis of their current system, gathering requirements from meetings and email correspondence, and object-oriented modelling, the project established a clear understanding of the problem domain and the functionalities needed in a new system. The resulting solution gives a clear overview of repair jobs through both a calendar and a list. Users can quickly add parts and services, and view key details for each repair, including the diagnostic, total price, and expected duration.

From a technical perspective, the project demonstrates how a multi-layered architecture, supported by frameworks such as *Spring Boot*, *PostgreSQL*, *Thymeleaf*, and *FullCalendar*, can be used to construct a functional web application. Implementation was supported by unit, integration, and user testing, which helped verify and validate system behavior, providing feedback for refinement.

The system fulfills all *must-have* and *should-have* requirements defined in the MoSCoW analysis, including the ability to create, edit, reschedule, and delete repairs, display repairs in both calendar and list views, add parts and services, calculate costs, and manage predefined product lists. These features were successfully implemented and validated in the testing phase. The *could-have* requirements, such as full Shopify integration, discount handling, and optimization for mobile view, were not implemented due to time constraints. Still, they remain realistic extensions for future development.

The project also brought forward several important reflections. The group encountered challenges in learning new technologies, structuring the application's layers, and integrating testing throughout the development process. Additionally, several potential improvements were identified, including the introduction of DTOs for cleaner API design, a product superclass to reduce code duplication, and an expanded emphasis on GDPR-compliant data handling. These insights form a valuable foundation for future iterations of the system.

Overall, the project resulted in a functioning repair management system that addresses Performsport's core workflow challenges. The solution provides good external value while leaving room for continued development, scalability, and integration with existing business systems.

11 Process Analysis

This chapter gives an overview of how we coordinated our work, the planning tools we used, and the workflow strategies that guided the project. We analyze how our group collaborated, how tasks were delegated and communicated, and how our development process evolved over time. We also discuss our experiences with both waterfall and agile approaches, reflect on the hybrid model we ended up using, and evaluate how our Scrum Master course has influenced our understanding of effective software development practices. Finally, we conclude with an evaluation of the process and improvements to consider for future projects.

11.1 Introduction

Throughout this semester project, our focus has not only been on designing and implementing a software solution for our client, Performsport, but also on improving our project management and collaborative workflow. As a group of software engineering students, we aim to continually refine our planning, communication, and project structure to become more skilled and versatile software engineers who can contribute to robust, practical, and high-quality solutions. This chapter presents the process behind the project, the decisions that shaped our workflow, and the lessons we learned regarding team collaboration and project-oriented work.

11.2 Development Approach - Waterfall VS Agile

During the first part of the semester, our development process was not yet guided by a specific methodology. It was only after approximately one month, when we were introduced to both the waterfall and agile approaches in the *System Development (SU)* course, that we began discussing which model would best suit our project. This caused a deeper reflection on how we wanted to structure our work going forward.

Later in the semester, all group members completed a two-day Scrum Master certification course and became Registered Scrum Masters in November 2025. This experience provided us with a more practical understanding of iterative development and the value of structured agile frameworks such as Scrum. Although this certification was introduced later in the project, it still influenced our approach to collaboration, planning, and incremental delivery for the remainder of the project and beyond.

11.2.1 Waterfall Characteristic in Our Workflow

Despite adopting agile elements later, several parts of our workflow followed a waterfall-like structure:

- We created early requirement artifacts such as the Rich Picture, FACTOR Analysis, MoSCoW Requirements, and later Use Cases, which formed the foundation for understanding the problem and defining the system's core functionality.
- We conducted architectural and design planning, including class diagrams, event tables, and structural/behavioral models, before implementing major parts of the system.

- Our project naturally followed a general phase order:

$$\text{Requirements} \rightarrow \text{Design} \rightarrow \text{Implementation} \rightarrow \text{Testing} \rightarrow \text{Evaluation}$$

These elements provided structure and helped the group maintain a clear overview during the early stages of the semester.

11.2.2 Agile Characteristics in Our Workflow

Even before we were formally introduced to agile development in the *System Development (SU)* course, our group naturally adopted several agile-like practices from the beginning of the semester. These practices became more intentional later, especially after completing the Scrum Master certification in November 2025, but the overall workflow was agile in spirit throughout the project:

- We planned in short, flexible 1-2 week intervals from the start, which allowed us to continuously adapt to new insights, feedback from our client, and changing priorities.
- Features were developed incrementally, evolving as our understanding of the system improved rather than being fully defined from the start.
- Scope, priorities, and design choices were adjusted continuously, especially when we discovered new constraints or better approaches during implementation.
- We used GitHub Projects to manage our programming tasks through a Kanban board, consisting of "To Do", "In Progress", and "Done" columns. Here, we could move tasks/ features across columns as they progress, providing an easy visual overview of the team's workflow.

These agile characteristics helped us react quickly to new information, refine features iteratively, and maintain a flexible development throughout the semester.

11.2.3 A Hybrid Model

In practice, our group ultimately adopted a hybrid approach that combined elements of both waterfall and agile methodologies:

Waterfall-like elements appeared in the early stages of the project, where we followed a more sequential process involving Rich Pictures, FACTOR Analysis, MoSCoW Requirements, Use Cases, and Architectural Design before beginning the majority of the coding.

Agile-inspired elements dominated the implementation phase, where we worked in short iterations, often adjusted plans, and focused on flexibility, incremental development, and continuous clarification of requirements.

This hybrid approach suited both the academic structure of the course and our own level of experience. However, we also recognized that applying agile principles more consistently, such as formalizing sprint planning, retrospectives, and a well-defined product backlog, could have improved clarity and coordination during implementation.

With all group members now Registered Scrum Masters, we expect to adopt a more structured Scrum-inspired workflow in future projects, including defined sprints, clearer roles, and regular reflection on teamwork and process improvement.

11.3 The Role of Scrum & Future Use

Although we did not formally implement Scrum during this project, the Scrum Master certification course that all group members completed played a meaningful role in shaping our understanding of structured agile development. The course provided practical insight into Scrum's core principles, roles, and events, and it helped us recognize similarities between Scrum and several practices we had already adopted, such as working in short intervals, collaborating frequently on campus, and refining features incrementally.

Although these activities were not formally organized as Scrum ceremonies, they reflected the underlying intent of Scrum, maintaining team alignment, promoting adaptability, and encouraging continuous improvement. Learning about Scrum also highlighted areas where our workflow could have been more systematic and efficient. For example, defined sprints, structured planning, and regular retrospectives could have provided clearer goals and more consistent feedback cycles during implementation.

Looking forward, we see clear value in adopting Scrum more intentionally in future projects. A structured sprint-based workflow, supported by a prioritized product backlog and recurring review and reflection sessions, would provide a strong framework for organizing our work. With all group members now certified Scrum Masters, we feel better equipped to incorporate these practices and expect them to contribute positively to both the development process and the quality of our future software projects.

11.4 Scheduling & Task Delegation

As mentioned in the previous sections, we implemented a sequential–agile hybrid approach for project planning, which required different types of scheduling. We used the scheduling application Trello to create a general timeline, including dates for major milestones, meetings with Performsport and our supervisor, and the planning of the documentation phase, which followed a waterfall structure.

Figure 24 below shows how a typical week during the documentation phase was planned using Trello. For each week, we documented the tasks we aimed to complete, meetings or other events involving Performsport (labelled in green), and meetings scheduled with our supervisor (labelled in purple).

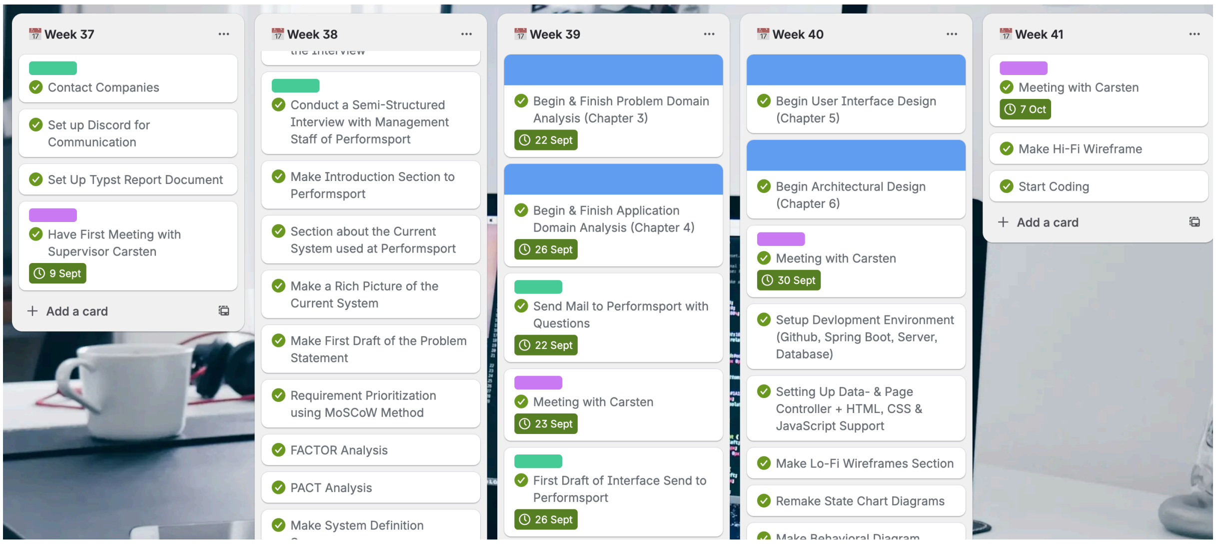


Figure 24: Schedule for the Documentation Phase of the Project using Trello

This way of planning the early stages of the project gave us a shared understanding of the direction we were taking and the tasks that needed to be completed. We did not assign tasks to specific group members at this stage, as we found it beneficial to work collectively on each task. This strengthened our shared understanding of every step of the process.

Later in the project, when we began writing code, we adopted a more agile approach, as described in Section 11.2.2. For this, we used a Kanban-style board on GitHub, shown in Figure 25. This worked well because it provided a structured overview of the tasks we needed to complete and allowed us to assign tasks to specific group members when appropriate.

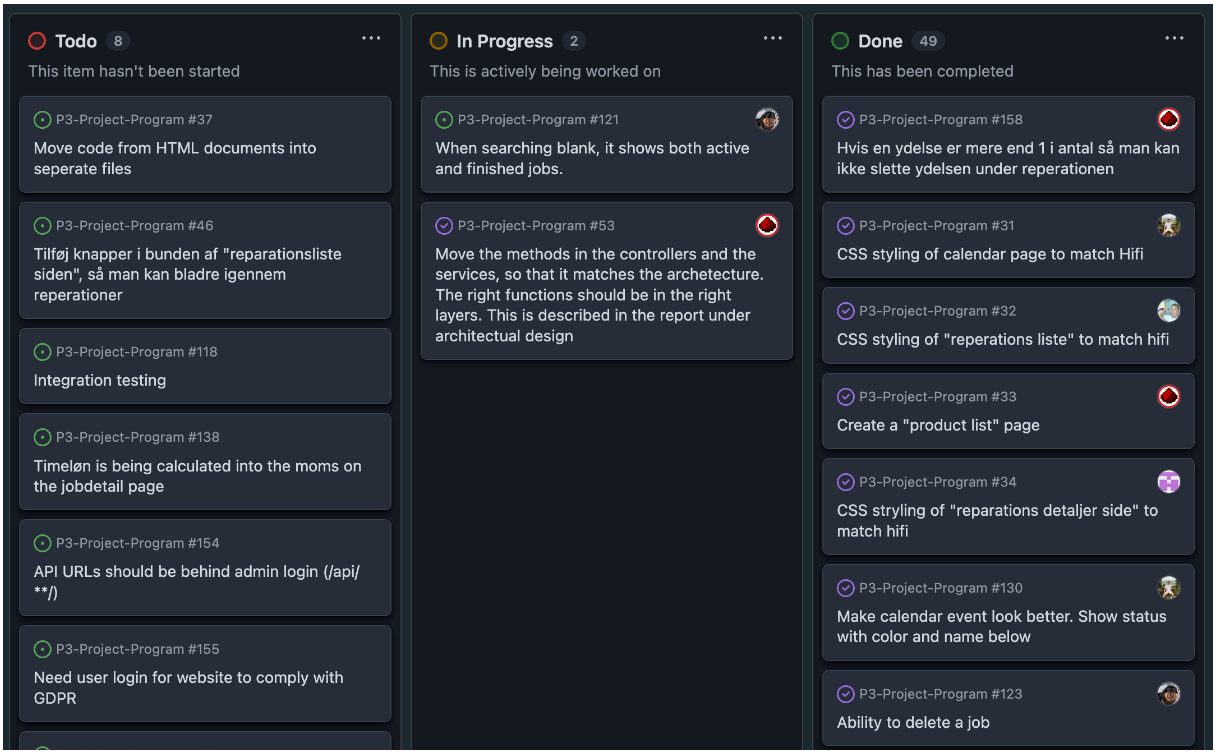


Figure 25: Kanban-style Board for Tracking Tasks using GitHub

Although we found the Kanban-style board useful, the insights gained from completing the Scrum course taught us that we could have used it more effectively, for example, as a tool for planning sprints during the agile phase of the project. Beyond this, the project taught us many valuable lessons, which are described in Section 11.7.

11.5 Communication

Communication has been an important aspect of our workflow throughout the projects, as effective collaboration relies on coordination, availability, and constant knowledge sharing. As a group, we prioritized working physically together on campus, as earlier experiences from our P2 project showed that in-person sessions significantly improved productivity and reduced misunderstandings. Physical presence allowed for quick discussion and problem-solving, and efficient solo and pair-programming, which led to faster progress overall.

Despite this, we maintained a flexible structure where members could participate remotely when necessary. In these cases, we coordinated work through Discord, which allowed us to join voice channels, share screens, and continue development in a structured online environment.

11.5.1 Communication Tools

Messenger

Messenger was used for quick and informal communication. This included small questions, daily updates, reminders, or clarifications regarding tasks. It also acted as our primary tool for urgent communication, such as changes in meeting times, sudden issues in the code, or notifications regarding absence. Messenger ensured everyone stayed updated and reachable throughout the project day-to-day.

Discord

Discord served as our main digital collaboration tool. When a group member(s) worked from home or when splitting into subgroups made sense, Discord was used for voice calls and screen sharing, allowing us to review code together and discuss solutions in real time. Files, links, and resources were also shared here, keeping conversations organized and easy to reference. Discord provided an efficient way to maintain progress even when not physically present on campus.

Together, Messenger and Discord ensured stable internal communication, regardless of whether we worked physically or remotely. These platforms made coordination smooth and allowed us to stay aligned on progress continuously.

11.5.2 Communication Flow Within the Project

The communication flow used in our group is illustrated in Figure 26, showing how daily internal coordination and supervisor feedback interact throughout the development process.

Our communication structure followed a simple cycle. Whether we met physically on campus or worked digitally through Discord, we typically began with a short alignment discussion, where we planned the tasks for the day and clarified who would work on what. Throughout the work sessions, communication happened often, either in person or through voice channels, allowing us to ask questions, solve issues quickly, and avoid blocking progress. This ongoing dialogue helped maintain momentum and ensured that everyone stayed aligned on goals and development direction.

Meetings with our supervisor were held every 1-2 weeks, depending on project needs rather than on a fixed weekly schedule. This approach was mutually agreed upon, as meetings were considered most valuable when we had concrete progress, questions, or work ready for evaluation. During these sessions, we received feedback, discussed architectural decisions, and clarified uncertainties. After each meeting, we followed up by implementing the feedback and adjusting the project direction when necessary.

This communication flow created a productive environment, allowing us to maintain momentum while ensuring access to guidance when needed. The combination of regular communication and flexible supervisor meetings contributed to an efficient and focused development process throughout the semester.

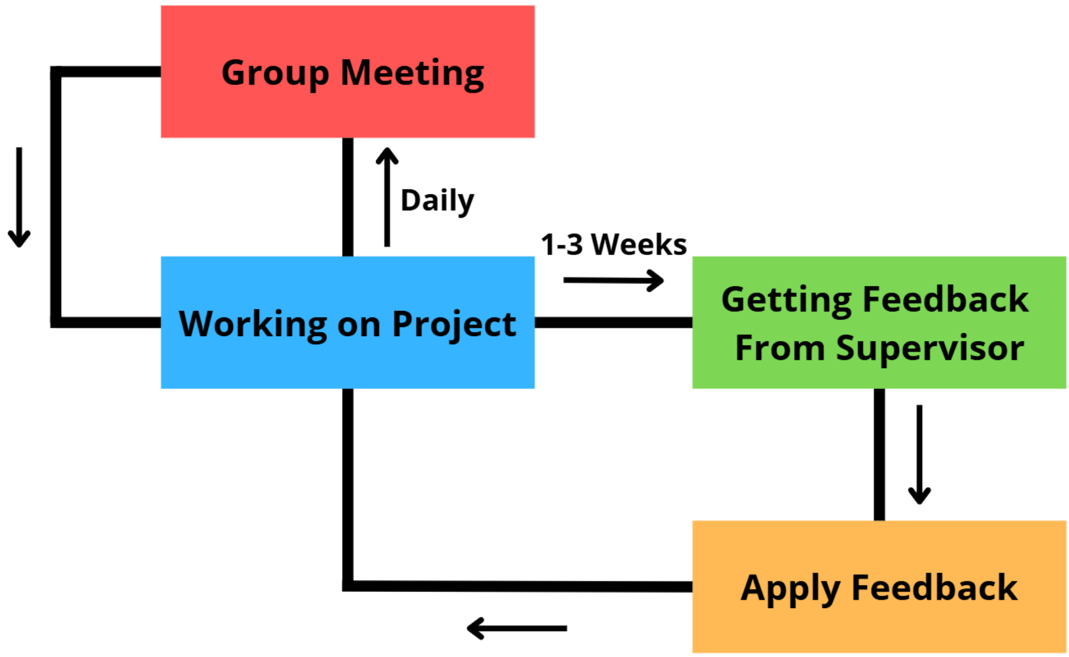


Figure 26: Illustration of Our Communication Flow During the Project

11.6 Risk Assessment

Throughout the project, several risks had the potential to impact both the process and the final product. This section highlights some of these risks, describes why they are important to identify and how we as a group can address these risks. The risks we found the most important are the technical risks, requirements risks, planning risks and team risks.

11.6.1 Technical Risks

One of the primary technical risks of the project was the choice of technologies to work with. When selecting frameworks such as Spring Boot, Docker, Thymeleaf, JUnit, and others, there was a risk that they might not integrate smoothly or might ultimately prove unsuitable for the solution. Since most of the group was initially unfamiliar with these technologies, the early stages of the project involved a steep learning curve as we worked to understand their functionality and limitations.

To mitigate this risk, we chose technologies that are widely used, well-documented, and known to work together in similar systems. We also researched these technologies, both before and during development, to ensure that our choices remained appropriate as the project evolved. This approach not only helped us select the right tools for the solution but also gave us valuable insight into when and how these technologies should be used, giving us experience that will benefit future projects.

Another technical risk was the potential accumulation of technical debt, which can occur when shortcuts are taken or when developers fail to follow best practices or proper design patterns. Due to the relatively small size and limited complexity of the system, technical debt did not become a significant concern during the project. However, we recognize that if the application were to be extended or new features added, certain parts of the codebase would need to be refactored to improve maintainability and make future development easier.

Overall, researching our technologies and reflecting on our implementation choices were some of the processes we learned the most from, and they provided us with practical experience that we can apply to future software projects.

11.6.2 Requirement Risks

Requirement risks were significant in this project because we worked with an external partner while also needing to meet academic expectations. There was a risk of misinterpreting requirements, as some descriptions from Performsport were initially unclear and not validated frequently enough. We also received relatively high-level requirements, which created uncertainty about whether we had enough information to implement the intended functionality. Looking back, we should have been better at reaching out to Performsport, when we had questions about the requirements.

Another challenge was that user involvement came too late. Since we conducted usability testing toward the end of development, we risked discovering requirement-related issues at a point where changes were more difficult to make. Balancing external value for Performsport with internal course requirements also created situations where priorities conflicted, such as Performsport prioritizing Shopify integration, while we assessed that it was outside the scope of the project.

A key takeaway is that future projects should involve the user more consistently. We should seek feedback earlier, validate assumptions more often, and ensure that requirements are defined with the user throughout the development process to reduce uncertainty and improve alignment with user needs.

11.6.3 Planning Risks

A central planning risk in this project was the difficulty of estimating deadlines at the beginning. Since this was our first time working with an external partner, it was challenging to predict when we would have a first prototype ready or when development needed to shift toward documentation. The unfamiliarity with both the problem domain and the technologies made early estimations uncertain and increased the chance of poor time allocation.

There was also a risk connected to task dependencies. Several parts of the system depended on others being completed first, meaning that delays in one area could prevent progress in others. This made accurate planning even more difficult, as one underestimated task could affect the entire timeline.

To address these risks, the project began with a waterfall-inspired planning phase to establish structure, main deliverables, and an overall timeline. Once implementation started, we adopted a more agile workflow, allowing us to adjust tasks, update priorities, and react to new information as it appeared. This combination helped reduce the impact of early estimation errors and allowed us to maintain progress even when plans had to be revised.

Overall, the project highlighted the value of combining structured planning with flexible, iterative development in this case, where documentation is valued highly.

11.6.4 Group Risks

Another risk we identified was what we called group risk, which relates to coordination and collaboration within the team. Since the project was developed by a group of students with varying levels of experience in the chosen technologies, there was a natural risk of uneven work distribution. This could lead to slowed progress or create dependencies on specific individuals who hold most of the knowledge about a particular technology or part of the code base.

Communication also posed a potential risk. Misunderstandings about requirements, task ownership, or implementation details could lead to duplicated work or inconsistencies in the code. Because the solution relies on many interconnected components, it was important that all group members had a shared understanding of both the overall structure of the application and the smaller design decisions made throughout development.

To mitigate these group-related risks, we held daily discussions about what each member was working on and openly talked through anything we did not fully understand. This was made easier by our decision to work physically together on campus, which helped us stay aligned and maintain a shared vision of the project.
Although this approach worked well, it could have been further supported by pair programming and Scrum-inspired activities such as structured daily meetings, more frequent reviews, and retrospectives to improve the workflow. This was an area where we learned a great deal as a group, as mentioned in Section 11.3, and these insights will serve us well in future projects.

11.7 Learning Experience

This semester significantly expanded our technical skills, both through coursework and through the practical requirements of developing a web application for Performsport. Many of the concepts and tools we worked with were new to us, and we gained a much clearer understanding of their relevance as the project progressed.

The *Object-Oriented Programming (OOP)* course was especially significant. Topics such as design principles and design patterns were introduced relatively late in the semester, at a point where most of our system had already been implemented, which meant we could not apply as many patterns as we ideally wanted. Still, we ended up using several, both intentionally and indirectly. For example, Spring's use of `ResponseEntity.ok().build()` demonstrates the Builder Pattern, and the framework itself relies heavily on patterns such as Dependency Injection (DI), Inversion of Control, Repository, and MVC. Our layered architecture with controllers, services, and repositories also reflects common architectural design patterns, even if we did not think of them as such during most of the development. Seeing these concepts later in the course helped us understand how our existing structure aligned with established design practices and where parts of the system, such as the `JobService` (Service Layer Class), could have benefited from a more pattern-oriented approach from the beginning.

We also worked with a more structured, multi-layered architecture than we had previously encountered. Although we had used simpler layered structures in earlier semesters, this was the first time we applied a full architectural separation with controllers, services, repositories, models, interfaces, and a technical layer. Understanding how these layers interact and why such separation matters, especially in larger systems, was a significant learning experience for the entire group.

Furthermore, we gained hands-on experience with modern frameworks and tools. Spring Boot introduced us to dependency injection and web-layer development, Bootstrap helped us design a responsive user interface, and FullCalendar enabled us to implement an interactive scheduling component. Working with these technologies showed us the practicality of using established frameworks to build advanced features more efficiently and reliably.

Overall, this semester gave us a challenging but rewarding learning experience. The experience of working with a larger architecture, new frameworks, and more complex system structures has strengthened our foundation and will significantly influence how we approach future projects.

11.8 Evaluation of the Process

Evaluating the overall process, the project gave us experience in planning, collaboration, and technical development. The hybrid structure we used, starting with a waterfall-inspired planning phase and later shifting toward a more agile workflow, proved effective for managing both documentation requirements and the uncertainties of working with an external partner. Early phase planning tools such as Trello helped establish a shared direction, while the GitHub Kanban board was used for more flexible task management during implementation.

Throughout the project, we learned how to coordinate more effectively as a group, communicate requirements clearly, and handle task dependencies. Working physically together on campus, combined with daily discussions, helped maintain alignment and reduce risks related to misunderstandings or uneven workload distribution.

The process also strengthened our technical capabilities. We gained practical experience with layered architecture, Spring Boot, Thymeleaf, and front-end frameworks (Bootstrap and Full-Calender). The introduction to Scrum later in the semester also helped us reflect on how our workflow could be more structured in future projects, especially through defined sprints, clearer backlog management, and regular retrospectives.

Overall, the project improved our ability to plan, adapt, and collaborate while developing a software solution. The insights gained, both technical and process-related, will influence how we approach future projects and help us work more efficiently.

12 Bibliography

[1] "Hvad er Shopify, og hvordan fungerer det? (2024) - Shopify Danmark." Accessed: Oct. 27, 2025. [Online]. Available: https://www.shopify.com/dk/blog/hvad-er-shopify

[2] "Spring boot home." Accessed: Sep. 29, 2025. [Online]. Available: https://spring.io/

[3] "MoSCoW Prioritization." Accessed: Sep. 29, 2025. [Online]. Available: https://www.productplan.com/glossary/moscow-prioritization/

[4] L. M. et al., *Object-oriented Analysis Design*. 2018.

[5] D. benyon, *Fourth Edition Designing User Experience A guide to HCI, UX and interaction design*. 2019.

[6] "Thymeleaf." Accessed: Nov. 04, 2025. [Online]. Available: https://www.thymeleaf.org/

[7] {and} B. M. O. contributors Jacob Thornton, "Bootstrap." Accessed: Nov. 04, 2025. [Online]. Available: https://getbootstrap.com/

[8] "How Spring Boot Application Works Internally?." Accessed: Oct. 27, 2025. [Online]. Available: https://www.geeksforgeeks.org/advance-java/how-spring-boot-application-works-internally/

[9] "Introduction to Spring Boot." Accessed: Oct. 27, 2025. [Online]. Available: https://www.geeksforgeeks.org/springboot/introduction-to-spring-boot/

[10] "Docker Docs." Accessed: Oct. 27, 2025. [Online]. Available: https://docs.docker.com/get-started/docker-overview/

[11] "What are ACID Transactions?." Accessed: Nov. 21, 2025. [Online]. Available: https://www.databricks.com/glossary/acid-transactions

[12] "Bootstrap Docs." Accessed: Nov. 17, 2025. [Online]. Available: https://getbootstrap.com/docs/5.3/

[13] D. Cookies, "The Ultimate Guide to Dependency Injection: Everything You Need to Know." Accessed: Dec. 08, 2025. [Online]. Available: https://devcookies.medium.com/the-ultimate-guide-to-dependency-injection-everything-you-need-to-know-7dfea8178739

[14] sudheeryadala, "Singleton Object Creation and Uses in Spring Boot." Accessed: Dec. 08, 2025. [Online]. Available: https://medium.com/@umasudheeryadala/singleton-object-creation-and-uses-in-spring-boot-e24170bd2d0e

[15] "Regulation (EU) GDPR." Accessed: Dec. 08, 2025. [Online]. Available: http://data.europa.eu/eli/reg/2016/679/oj

[16] R. & Chisnell, "Rubin & Chisnell - Handbook of Usability Testing," pp. 27–37 & 65–91, 1994.

13 Appendix

13.1 First Interview with Performsport

13.1.1 Interview Notes - Semi-Structured Interview (Translated to English)

They would like a list of different types of repairs to choose from when booking an appointment.
- Each repair type should have a default time and price that can be edited in a menu or list.

Each work order/booking should contain the following information:
- Bike type
- Customer
- Phone number
- Type of repair
- Note
- Date
- Additional services or bike parts (selected from a list)

If possible, it would be nice to pull customers from Shopify so they can be selected when booking.

They want to be able to search for customers by phone number, and possibly also by name.

When they book a time slot, it should block time in their calendar.

A repair should have a status that can be updated as the bike is delivered and work progresses.
- This status can be supported by color coding.

They want to quickly see whether the bike for a repair is a the shop or not yet delivered, e.g., with a color indicator.

Example statuses:
- Not arrived
- Arrived
- In progress
- Finished
- Waiting for parts, etc.

They need to track if a repair is missing a part (e.g., a cassette that is not in stock).

When the status is changed to “Finished”, there should be an option to automatically send an SMS to the customer.
- A pop-up could prompt them to send this message.

Priority: The calendar is the most important feature.
- Shopify Integration is very nice to have, but not the first priority.

Current Workflow

- Create a draft order in Shopify (name, phone, description of the work).
- Attach a tag/note to the bike.
- Add parts to the draft order once the bike is finished, along with labor hours.
- Notify the customer that the bike is ready.
- Customers picks up the bike.
- Convert the draft to a final sale in Shopify.
- They are unsure whether they will continue to create the customer in Shopify first or start by creating the repair in the new system.
  - They want to see what is possible before deciding which workflow is best.

13.2 System Definition with PACT

The PACT framework examines People, Activities, Context, and Technologies and will be used to understand how a system should be designed and implemented.

13.2.1 People

Staff members, including mechanics and other employees, are responsible for managing bike jobs, updating repair statuses, handling inventory, and maintaining communication with customers. Some staff members are English speakers and require an interface written in English.

Management has a supervisory role over the company, and thus has an interest in ensuring that the workflow are efficient and that the calender is filled out to match the number of mechanics at work.

13.2.2 Activities

Staff will use the system for creating jobs and adding them to a calendar. Customers will not interact with the system. Some activities include creating, editing and changing the status of repair jobs and editing the list of spare parts in the system if needed.

13.2.3 Contexts

Shop environment in Aalborg, where staff interact directly with customers and perform the repairs on the bikes.

The repair service is currently tied to the Shopify system, which is primarily designed for retail sales rather than service management. This has resulted in a disconnected workflow that combines digital and manual processes.

The company and the management team values great personal service and reliability, meaning that the staff need the proper tools and systems that allow them to provide a smooth and reliable repair experience.

13.2.4 Technologies

The project will be implemented mainly in Java as the programming language, java was chosen on the basis of our course OOP and its strong support for the object-oriented paradigm.

Input Technologies

The system will use keyboard and mouse to handle user interaction.

Output Technologies

The system uses the user's screen as the main output technology. SMS notifications are also used as an additional output channel to send customers important updates and reminders outside the application.

13.3 Testing Plan

Structured plan/outline of the testing process:

1. **Research Questions:** How easily can users perform core tasks? Which parts of the interface cause confusion or errors? Do users understand specific features and workflow?
2. **Participant Characteristics:** Describe who, the number of people, relevant technical abilities, and maybe ask the group in the group room or employees of Performsport.
3. **Method/Test Design:** Test could be assessment-type with participants performing all tasks, while being moderated synchronously, but some tasks could be adapted to an asynchronous format for remote feedback.
4. **Task List:** Representative tasks assigning should be well explained and should include: Task name, Brief description, Success criteria (Optional), and a timing benchmark (Optional).
5. **Data to Be Collected:**
   - **Quantitative Performance Data**:
     Success/failure, completion time, number of errors, need for assistance.
   - **Qualitative Preference Data**:
     User comments, satisfaction ratings, perceived ease of use.
6. **Analysis and Presentation of Results**:
   - **Quantitative performance data** will be analyzed to identify patterns in user performance and behavior, which will be summarized in simple tables for clarity.
   - **Qualitative preference data/insights** will be grouped by recurring themes to highlight common issues or strengths of the application. [16]

13.3.1 User Tests (English Version)

Overview

Estimated Duration: ~15 minutes.

Assignment 1: Create Repair

Description:

On the calender main page figure out a way to create a scheduled repair job.

Succes Criteria:

Spot your scheduled repair job and confirm correct time in calender.

Note: *Should be timed*

Assignment 2: Edit Repair

Description:

Locate your scheduled repair job and edit it by modifying each input field slightly.

Success Criteria:

You can find your updated repair job at its new time/location in the calendar and confirm that the changes were applied.

Note: *Should be timed.*

Assignment 3: Repair List
Description:
From within the "Reparationsliste" page, create a repair with the status "ikke indleveret". Check the differences in the available filters. Then change the repair status to "afhentet" and observe how it appears under the Active/Completed filters.

Success Criteria:
You can locate your repair job under the correct filters at each step.

Note: *Should be timed.*

Assignment 4: Product List
Description:
From the product list page, add a service ("ydelse") with all fields filled out, and link it to one of your previous repairs in the Reparationsliste.

Success Criteria:
You can find your service added to the repair, and you can confirm that the total price has been recalculated correctly.

Note: *Should be timed.*

General Feedback
Provide concise overall feedback on the workflows of the pages.

13.3.2 User Tests (Danish Version)

Oversigt
Forventet Varighed: ~15 minutter.

Opgave 1: Opret Reparation
Beskrivelse:
På kalenderens hovedside skal du finde en måde at oprette en planlagt reparation.

Succeskriterier:
Du kan finde din planlagte reparation i kalenderen og bekræfte tidspunktet.

Note: *Skal times.*

Opgave 2: Rediger Reparation
Beskrivelse:
Find din planlagte reparation og redigér det ved at ændre alle inputfelter en smule.

Succeskriterier:
Du kan finde din reparations det nye sted i kalenderen og bekræfte ændringerne.

Note: *Skal times.*

Opgave 3: Reparationsliste
Beskrivelse:
Opret en reparation via siden "Reparationsliste" med status "ikke indleveret". Undersøg forskellen i filtrene. Ændre derefter status til "afhentet", og se hvordan reparationen flytter sig mellem Aktive/Afsluttede filtrene.

Succeskriterier:
Du kan finde din reparation i de relevante filtre på hvert trin.

Note: *Skal times.*

Opgave 4: Produktliste
Beskrivelse:
Under produktlisten tilføjes en ydelse med alle felter udfyldt. Tilknyt ydelsen til en af dine tidligere reparationer via reparationslisten.

Succeskriterier:
Du kan finde ydelsen på reparationen, og den opdaterede pris vises korrekt.

Note: *Skal times.*

Generel Feedback
Giv kort, overordnet feedback på workflowet på siderne.

Table 9: Data Types